MODUL 11B · PROYEK HIVE CRUD

Proyek CRUD Mahasiswa: Hive + GetX + Pola MVC

Membangun aplikasi CRUD Data Mahasiswa dari proyek kosong sampai jadi: model dan TypeAdapter manual, HiveService sebagai lapisan akses data, MahasiswaController (GetX) dengan state reaktif, validasi, cegah NIM ganda, dan dialog hapus, Binding dan named routes, view daftar dengan pencarian serta satu form untuk tambah dan edit, unit test dan widget test, listing lengkap siap salin, dan troubleshooting.

Durasi Pertemuan 11 (lanjutan, 150 Menit) + Belajar Mandiri

Prasyarat Modul 05: Form & Validasi
Modul 08: State Management dengan GetX + MVC
Modul 11: Database NoSQL Lokal dengan Hive

Capaian Pembelajaran

- Menyusun proyek Flutter dengan struktur folder MVC: models, services, controllers, bindings, routes, dan views.
- Menulis model immutable dengan copyWith dan TypeAdapter manual dengan urutan read/write yang konsisten.
- Membuat service yang membungkus Box Hive untuk operasi create, read, update, dan delete.
- Mengelola state reaktif (RxList, RxString), validasi form, pencarian, dan mode tambah/edit di satu GetxController.
- Mendaftarkan controller lewat Bindings dan berpindah halaman dengan named routes GetX.
- Membangun View tipis dengan GetView dan Obx yang tidak pernah mengakses Hive secara langsung.
- Menguji service Hive dengan unit test dan alur CRUD lewat UI dengan widget test.

Daftar Isi Modul

1. Gambaran Proyek & Arsitektur MVC
2. Membuat Proyek & Memasang Package
3. Model Mahasiswa
4. TypeAdapter Manual
5. HiveService: Lapisan Akses Data
6. MahasiswaController: State & Logika CRUD
7. Routes & Binding
8. View Daftar Mahasiswa (Read, Cari, Hapus)

9. View Form (Create & Update)
10. main.dart: Inisialisasi Hive & GetMaterialApp
11. Menjalankan & Menguji Alur CRUD
12. Unit Test & Widget Test
13. Listing Lengkap (Siap Copy-Paste)
14. Kesalahan Umum & Ringkasan
15. Tugas Praktikum
16. Kuis Pemahaman & Kunci Jawaban

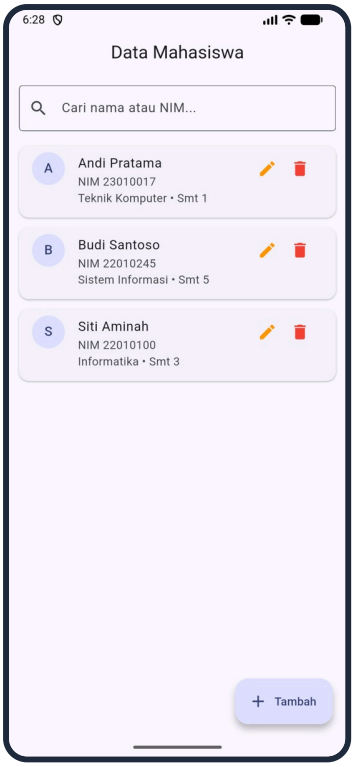
1. Gambaran Proyek & Arsitektur MVC

Di modul ini kita membangun satu aplikasi utuh dari proyek kosong: **CRUD Data Mahasiswa**. Data disimpan di **Hive** (database lokal, jalan tanpa internet), state dan navigasi diatur oleh **GetX**, dan kodenya disusun dengan pola **MVC** (Model, View, Controller). Setiap file ditulis lengkap, jadi Anda bisa menyalinnya satu per satu lalu langsung menjalankannya.

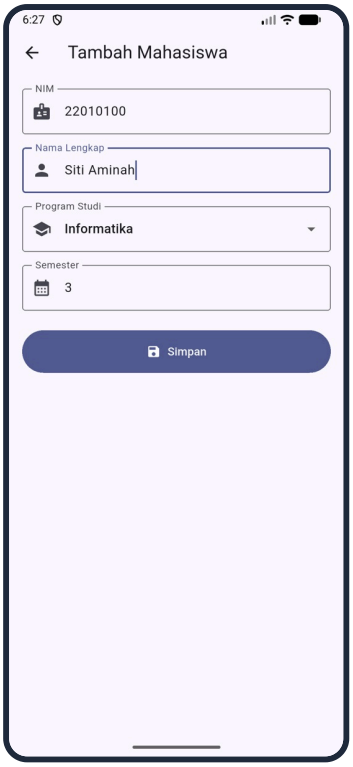
Fitur aplikasi

Operasi	Yang dilakukan pengguna	Method di controller
Create	Tekan tombol *Tambah*, isi NIM, nama, prodi, semester, lalu *Simpan*	`openAddForm()`, `save()`
Read	Melihat daftar mahasiswa yang diurutkan berdasarkan nama	`loadData()`
Update	Tekan ikon pensil, ubah data, lalu *Perbarui*	`openEditForm()`, `save()`
Delete	Tekan ikon tempat sampah, lalu konfirmasi *Hapus*	`confirmDelete()`
Cari	Mengetik nama atau NIM di kolom pencarian	`keyword` + `filteredList`
Validasi	NIM wajib angka minimal 8 digit, nama wajib, semester 1-14, NIM tidak boleh ganda	`validateNim()` dan kawan-kawan

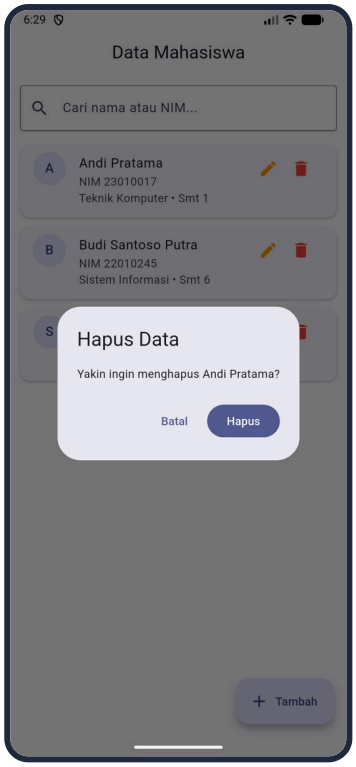
Tampilan akhir aplikasi yang akan kita buat:



Halaman daftar (Read)



Form tambah (Create)



Konfirmasi hapus (Delete)

Apa itu MVC di proyek ini?

MVC membagi kode menjadi tiga peran supaya setiap file punya satu tugas yang jelas:

- **Model (M)**: bentuk data dan cara menyimpannya. Di sini: class `Mahasiswa`, `MahasiswaAdapter` (cara Hive mengubah objek menjadi byte), dan `HiveService` (satu-satunya kelas yang menyentuh database).
- **View (V)**: tampilan. Di sini: `MahasiswaListView` dan `MahasiswaFormView`. View hanya menampilkan data dan meneruskan klik ke controller. View **tidak pernah** memanggil Hive.
- **Controller (C)**: otak aplikasi. `MahasiswaController` menyimpan state reaktif (`.obs`), memvalidasi input, dan memanggil `HiveService` untuk tambah, ubah, dan hapus.

Dua bagian pendukung dari GetX: **Routes** (daftar nama halaman) dan **Binding** (tempat mendaftarkan controller sebelum halaman dibuka).

Alur data (arah panah satu jalur)

```
┌──────────────────────────────┐
| VIEW                         |  MahasiswaListView, MahasiswaFormView
| Obx(() => ...) membaca state |  tombol memanggil controller.save(), dst.
└──────────────┬───────────────┘
               | memanggil method
               ▼
┌──────────────────────────────┐
| CONTROLLER                   |  MahasiswaController (GetxController)
| mahasiswaList.obs, keyword   |  validasi, navigasi, snackbar, dialog
└──────────────┬───────────────┘
               | create / getAll / update / delete
               ▼
```

```
┌──────────────────────────────┐
| MODEL                        |  HiveService  ──►  Box<Mahasiswa> 'mahasiswa_box'
|                              |  Mahasiswa + MahasiswaAdapter (typeId 0)
└──────────────────────────────┘
```

Contoh saat pengguna menekan *Simpan*:

1. View memanggil `controller.save()`.
2. Controller memvalidasi form, membuat objek `Mahasiswa`, lalu memanggil `_service.create(mhs)`.
3. `HiveService` menulis objek ke box. `MahasiswaAdapter` mengubahnya menjadi byte di file.
4. Controller memanggil `loadData()` yang mengisi ulang `mahasiswaList` (sebuah `RxList`).
5. Karena `mahasiswaList` berubah, `Obx` di halaman daftar otomatis membangun ulang tampilannya. Tidak ada `setState` sama sekali.

Struktur folder akhir

```
crud_mahasiswa/
├── pubspec.yaml
├── lib/
|   ├── main.dart                     # init Hive + GetMaterialApp
|   ├── models/
|   |   ├── mahasiswa_model.dart      # M: class Mahasiswa
|   |   └── mahasiswa_adapter.dart    # M: TypeAdapter untuk Hive
|   ├── services/
|   |   └── hive_service.dart         # M: akses database Hive (CRUD)
|   ├── controllers/
|   |   └── mahasiswa_controller.dart  # C: state reaktif + logika CRUD
|   ├── bindings/
|   |   └── mahasiswa_binding.dart    # registrasi controller (GetX)
|   ├── routes/
|   |   ├── app_routes.dart           # nama-nama route
|   |   └── app_pages.dart            # route -> halaman + binding
|   └── views/
|       ├── mahasiswa_list_view.dart   # V: daftar, cari, edit, hapus
|       └── mahasiswa_form_view.dart   # V: form tambah & edit
└── test/
    ├── hive_service_test.dart        # unit test CRUD Hive
    └── crud_flow_test.dart           # widget test alur CRUD lewat UI
```

> 💡 **Cara mengikuti modul ini**
>
> Kerjakan section 2 sampai 10 **berurutan**, dari lapisan paling bawah (model) ke paling atas (view dan `main.dart`). Dengan urutan ini setiap file yang Anda tulis hanya bergantung pada file yang sudah ada. Garis merah di editor yang muncul karena file belum dibuat akan hilang sendiri setelah file tersebut ditulis. Jika ingin langsung jadi, buka section 13 (Listing Lengkap) dan salin semua file sekaligus.

ℹ️ **Prasyarat**

Modul ini merangkum materi Modul 05 (Form & validasi), Modul 08 (GetX & MVC), dan Modul 11 (Hive). Jika ada istilah yang terasa asing, misalnya `.obs`, `Obx`, `Bindings`, atau `TypeAdapter`, buka lagi modul tersebut.

2. Membuat Proyek & Memasang Package

Langkah pertama: membuat proyek Flutter baru, memasang tiga package, lalu menyiapkan folder MVC.

1. Buat proyek

Buka terminal di folder tempat Anda menyimpan proyek, lalu jalankan:

```
flutter create crud_mahasiswa
cd crud_mahasiswa
```

> ⚠️ **Nama proyek harus `crud_mahasiswa`**
>
> File test di section 12 mengimpor `package:crud_mahasiswa/...`. Nama package diambil dari nama proyek, jadi jika Anda memakai nama lain, ganti juga bagian import di file test.

2. Pasang package

```
flutter pub add get hive_ce hive_ce_flutter
```

Package	Kegunaan di proyek ini
`get`	State reaktif (`.obs`, `Obx`), dependency injection (`Bindings`, `GetView`), named route, snackbar, dan dialog
`hive_ce`	Database NoSQL lokal: `Box`, `TypeAdapter`, `put`, `delete`
`hive_ce_flutter`	`Hive.initFlutter()` yang otomatis memilih folder penyimpanan di HP

Setelah perintah di atas, bagian `dependencies` di `pubspec.yaml` kira-kira menjadi seperti ini (nomor versi di laptop Anda boleh lebih baru):

`pubspec.yaml` (bagian dependencies)

```yaml
dependencies:
  flutter:
    sdk: flutter

  cupertino_icons: ^1.0.8
  get: ^4.7.3
  hive_ce: ^2.23.3
  hive_ce_flutter: ^2.5.0
```

ℹ️ **Kenapa `hive_ce` , bukan `hive` ?**

Package `hive` versi 2.x sudah tidak dirawat lagi oleh pembuatnya. `hive_ce` (Community Edition) adalah lanjutannya dengan API yang sama persis, seperti dibahas di Modul 11. Jika di laboratorium Anda terpaksa memakai `hive` + `hive_flutter` lama, kode di modul ini tetap berjalan. Cukup ganti baris import di `main.dart` , `mahasiswa_adapter.dart` , `hive_service.dart` , dan kedua file test:

```
// Versi modul ini (hive_ce)
import 'package:hive_ce/hive_ce.dart';
import 'package:hive_ce_flutter/hive_ce_flutter.dart';

// Versi package lama (hive)
import 'package:hive/hive.dart';
import 'package:hive_flutter/hive_flutter.dart';
```

3. Siapkan folder MVC dan hapus test bawaan

```
# macOS / Linux / Git Bash
mkdir -p lib/models lib/services lib/controllers lib/bindings lib/routes lib/views
rm test/widget_test.dart
```

```
# Windows PowerShell
mkdir lib/models, lib/services, lib/controllers, lib/bindings, lib/routes, lib/views
Remove-Item test/widget_test.dart
```

File `test/widget_test.dart` bawaan menguji aplikasi counter. Karena aplikasi counter akan kita ganti, test itu pasti gagal, jadi kita hapus. Test baru ditulis di section 12.

Isi `lib/main.dart` biarkan dulu. Kita menggantinya di section 10 setelah semua file lain siap.

💡 **Cek cepat**

Jalankan `flutter pub get` lalu pastikan tidak ada pesan error. Jika muncul *version solving failed*, jalankan `flutter upgrade` agar SDK Flutter Anda versi stabil terbaru, lalu ulangi `flutter pub add` .

3. Model Mahasiswa

Kita mulai dari lapisan paling bawah: **Model**. Model adalah class Dart biasa yang mendeskripsikan bentuk satu data mahasiswa. Class ini tidak tahu apa-apa tentang tampilan, GetX, maupun Hive.

Buat file baru `lib/models/mahasiswa_model.dart`, lalu salin kode berikut:

`lib/models/mahasiswa_model.dart`

```dart
/// MODEL: representasi data Mahasiswa.
/// Class ini murni berisi data (tidak tahu soal UI maupun GetX).
class Mahasiswa {
  final String id;
  final String nim;
  final String nama;
  final String prodi;
  final int semester;

  Mahasiswa({
    required this.id,
    required this.nim,
    required this.nama,
    required this.prodi,
    required this.semester,
  });

  /// Membuat salinan objek dengan sebagian field yang diubah.
  Mahasiswa copyWith({
    String? nim,
    String? nama,
    String? prodi,
    int? semester,
  }) {
    return Mahasiswa(
      id: id,
      nim: nim ?? this.nim,
      nama: nama ?? this.nama,
      prodi: prodi ?? this.prodi,
      semester: semester ?? this.semester,
    );
  }
}
```

Penjelasan kode

Field	Tipe	Keterangan
`id`	`String`	Penanda unik setiap data. Dipakai sebagai **key** di box Hive. Dibuat otomatis oleh controller, pengguna tidak pernah melihatnya.
`nim`	`String`	Nomor induk mahasiswa. Disimpan sebagai teks (bukan `int`) karena NIM bisa diawali angka 0 dan tidak pernah dihitung.
`nama`	`String`	Nama lengkap.
`prodi`	`String`	Program studi, dipilih dari dropdown.
`semester`	`int`	Semester 1 sampai 14. Bertipe angka agar mudah dibandingkan dan divalidasi.

- **Semua field `final`**: setelah objek dibuat, isinya tidak bisa diubah. Objek seperti ini disebut *immutable*. Keuntungannya, data tidak bisa berubah diam-diam dari tempat lain sehingga bug lebih mudah dilacak.
- **`required`** di constructor memaksa semua field diisi. Lupa mengisi satu field langsung menjadi error saat kompilasi, bukan saat aplikasi berjalan.
- **`copyWith`**: karena objek tidak bisa diubah, cara "mengedit" adalah membuat salinan dengan sebagian field diganti. Field yang tidak dikirim (`null`) memakai nilai lama berkat operator `??`. Perhatikan bahwa `id` sengaja tidak bisa diganti, karena id adalah identitas data.

Contoh penggunaan `copyWith` (dipakai di unit test section 12):

```dart
final budi = Mahasiswa(
  id: '1', nim: '220101001', nama: 'Budi', prodi: 'Informatika', semester: 5,
);

final budiBaru = budi.copyWith(nama: 'Budi Santoso', semester: 6);
// budiBaru.id     -> '1'            (tetap)
// budiBaru.nim    -> '220101001'    (tetap)
// budiBaru.nama   -> 'Budi Santoso' (diganti)
// budiBaru.semester -> 6            (diganti)
// budi.nama       -> 'Budi'         (objek lama tidak berubah)
```

ℹ️ **Kenapa model tidak memakai anotasi Hive?**

Di Modul 11 Anda melihat dua cara: TypeAdapter manual dan adapter hasil generate (`@GenerateAdapters`). Proyek ini memakai cara **manual** agar model tetap bersih dan Anda melihat sendiri bagaimana Hive menyimpan data. Kode Hive-nya dipisah ke file adapter di section berikutnya.

4. TypeAdapter Manual

Hive hanya mengerti tipe dasar seperti `String`, `int`, `bool`, `List`, dan `Map`. Agar Hive bisa menyimpan objek `Mahasiswa`, kita perlu memberi "kamus terjemahan" yang disebut **TypeAdapter**: bagaimana objek ditulis menjadi byte (`write`) dan bagaimana byte dibaca kembali menjadi objek (`read`).

Buat file `lib/models/mahasiswa_adapter.dart`:

`lib/models/mahasiswa_adapter.dart`

```dart
import 'package:hive_ce/hive_ce.dart';

import 'mahasiswa_model.dart';

/// TypeAdapter memberi tahu Hive cara menyimpan (write)
/// dan membaca (read) objek Mahasiswa dalam bentuk biner.
///
/// ATURAN PENTING:
/// 1. typeId harus unik per class (0 - 223).
/// 2. Urutan read HARUS sama persis dengan urutan write.
class MahasiswaAdapter extends TypeAdapter<Mahasiswa> {
  @override
  final int typeId = 0;

  @override
  Mahasiswa read(BinaryReader reader) {
    return Mahasiswa(
      id: reader.readString(),
      nim: reader.readString(),
      nama: reader.readString(),
      prodi: reader.readString(),
      semester: reader.readInt(),
    );
  }

  @override
  void write(BinaryWriter writer, Mahasiswa obj) {
    writer.writeString(obj.id);
    writer.writeString(obj.nim);
    writer.writeString(obj.nama);
    writer.writeString(obj.prodi);
    writer.writeInt(obj.semester);
  }
}
```

Cara kerja adapter

Bayangkan Hive menulis data ke file seperti menulis di satu baris panjang tanpa nama kolom. Saat menyimpan Siti Aminah, isi file kira-kira seperti ini:

```
[typeId 0] "1731300000000" "22010100" "Siti Aminah" "Informatika" 3
           └─ id ──────┘  └─ nim ─┘  └─ nama ────┘ └─ prodi ──┘ └ semester
```

- `typeId = 0`: Hive menulis angka ini di depan setiap objek. Saat membaca, Hive melihat angka 0 lalu tahu harus memakai `MahasiswaAdapter`. Setiap class yang punya adapter wajib memakai `typeId` berbeda (0 sampai 223).
- `write` menulis kelima field secara berurutan: `id`, `nim`, `nama`, `prodi` (semuanya `writeString`), lalu `semester` (`writeInt`).
- `read` membaca dengan urutan dan tipe yang **sama persis**. Karena tidak ada nama kolom, satu-satunya penanda field adalah urutannya.

> ⚠️ **Dua aturan yang tidak boleh dilanggar**
>
> 1. **Urutan `read` = urutan `write`.** Jika `write` menulis nim lalu nama, tetapi `read` membaca nama lalu nim, data akan tertukar. Jika tipenya berbeda (misalnya `readInt` untuk data yang ditulis `writeString`), aplikasi akan crash saat membuka box.
> 2. **Jangan mengubah `typeId`** setelah ada data tersimpan. Data lama tercatat dengan id 0; jika adapter diganti menjadi 1, Hive tidak menemukan adapter untuk data lama.
>
> Jika selama latihan Anda terlanjur mengubah urutan field dan aplikasi crash, cara tercepat adalah **uninstall aplikasi** dari emulator agar file box lama terhapus, lalu jalankan ulang. Cara aman menambah field tanpa menghapus data dibahas di Modul 11 section 5 (byte versi).

5. HiveService: Lapisan Akses Data

`HiveService` adalah **satu-satunya** class yang berbicara langsung dengan Hive. Controller cukup memanggil `create`, `getAll`, `update`, atau `delete` tanpa perlu tahu nama box maupun cara Hive bekerja.

Buat file `lib/services/hive_service.dart`:

`lib/services/hive_service.dart`

```dart
import 'package:hive_ce/hive_ce.dart';

import '../models/mahasiswa_model.dart';

/// Lapisan akses data (bagian dari MODEL pada MVC).
/// Semua operasi CRUD ke Hive dikumpulkan di sini,
/// sehingga Controller tidak perlu tahu detail database.
class HiveService {
  static const String boxName = 'mahasiswa_box';

  Box<Mahasiswa> get _box => Hive.box<Mahasiswa>(boxName);

  // READ
  List<Mahasiswa> getAll() => _box.values.toList();

  // CREATE (key = id mahasiswa)
  Future<void> create(Mahasiswa mhs) => _box.put(mhs.id, mhs);

  // UPDATE (put dengan key yang sama akan menimpa data lama)
  Future<void> update(Mahasiswa mhs) => _box.put(mhs.id, mhs);

  // DELETE
  Future<void> delete(String id) => _box.delete(id);
}
```

Penjelasan per baris

Kode	Arti
`static const String boxName = 'mahasiswa_box';`	Nama box disimpan di satu tempat. `main.dart` dan file test memakai `HiveService.boxName`, sehingga tidak ada salah ketik nama box.
`Box<Mahasiswa> get _box => Hive.box<Mahasiswa>(boxName);`	Mengambil box yang **sudah dibuka** di `main()`. `Hive.box()` bersifat sinkron dan hanya boleh dipanggil setelah `Hive.openBox()` selesai.
`_box.values.toList()`	**READ**: semua objek `Mahasiswa` di box dalam bentuk `List`.
`_box.put(mhs.id, mhs)`	**CREATE**: menyimpan objek dengan key = `id`. Jika key belum ada, data baru dibuat.
`_box.put(mhs.id, mhs)` (lagi)	**UPDATE**: kode yang sama! Jika key sudah ada, `put` menimpa data lama. Inilah alasan id dipakai sebagai key.
`_box.delete(id)`	**DELETE**: menghapus data berdasarkan key.

Jika dibandingkan dengan SQL di Modul 10, kira-kira seperti ini:

Operasi	SQLite (Modul 10)	Hive (proyek ini)
Create	`INSERT INTO mahasiswa ...`	`box.put(id, mhs)`
Read	`SELECT * FROM mahasiswa`	`box.values.toList()`
Update	`UPDATE mahasiswa SET ... WHERE id = ?`	`box.put(id, mhsBaru)`
Delete	`DELETE FROM mahasiswa WHERE id = ?`	`box.delete(id)`

💡 **Kenapa create dan update dipisah kalau isinya sama?**

Supaya kode controller terbaca jelas: `_service.create(mhs)` dan `_service.update(mhs)` langsung memberi tahu maksudnya. Jika suatu saat penyimpanan diganti ke Supabase (Modul 12), create menjadi `insert` dan update menjadi `update`. Cukup isi `HiveService` yang diubah, controller dan view tidak perlu disentuh.

ℹ️ **`Future<void>`**

`put` dan `delete` menulis ke file sehingga butuh waktu sesaat. Keduanya mengembalikan `Future`, dan controller memanggilnya dengan `await` agar data benar-benar tersimpan sebelum daftar dimuat ulang. Sebaliknya `getAll()` sinkron karena Hive menyimpan salinan isi box di memori.

6. MahasiswaController: State & Logika CRUD

`MahasiswaController` adalah inti aplikasi. Semua yang "berpikir" ada di sini: menyimpan daftar mahasiswa sebagai state reaktif, menyaring pencarian, membuka form tambah/edit, memvalidasi input, menyimpan, dan menghapus. View cukup menampilkan isi controller dan memanggil method-nya.

Buat file `lib/controllers/mahasiswa_controller.dart`. File ini paling panjang, jadi salin utuh dulu lalu baca penjelasan per bagian di bawahnya:

`lib/controllers/mahasiswa_controller.dart`

```dart
import 'package:flutter/material.dart';
import 'package:get/get.dart';

import '../models/mahasiswa_model.dart';
import '../routes/app_routes.dart';
import '../services/hive_service.dart';

/// CONTROLLER: penghubung antara View dan Model.
/// Menyimpan state (reaktif) dan berisi seluruh logika CRUD.
class MahasiswaController extends GetxController {
  final HiveService _service = HiveService();

  // ------------------ STATE REAKTIF ------------------
  final mahasiswaList = <Mahasiswa>[].obs;
  final keyword = ''.obs;
  final selectedProdi = 'Informatika'.obs;

  final List<String> daftarProdi = const [
    'Informatika',
    'Sistem Informasi',
    'Teknik Komputer',
    'Data Science',
  ];

  // ------------------ STATE FORM ------------------
  final formKey = GlobalKey<FormState>();
  final nimC = TextEditingController();
  final namaC = TextEditingController();
  final semesterC = TextEditingController();

  /// null  -> mode TAMBAH
  /// berisi id -> mode EDIT
  String? editingId;
  bool get isEdit => editingId != null;

  /// Data yang ditampilkan (sudah difilter pencarian).
  List<Mahasiswa> get filteredList {
    final q = keyword.value.toLowerCase();
    if (q.isEmpty) return mahasiswaList;
    return mahasiswaList
        .where((m) =>
            m.nama.toLowerCase().contains(q) || m.nim.contains(q))
        .toList();
```

```
  }

  @override
  void onInit() {
    super.onInit();
    loadData();
  }

  // ------------------ READ ------------------
  void loadData() {
    final data = _service.getAll()
      ..sort((a, b) => a.nama.toLowerCase().compareTo(b.nama.toLowerCase()));
    mahasiswaList.assignAll(data);
  }

  // ------------------ NAVIGASI KE FORM ------------------
  void openAddForm() {
    editingId = null;
    nimC.clear();
    namaC.clear();
    semesterC.clear();
    selectedProdi.value = daftarProdi.first;
    Get.toNamed(AppRoutes.form);
  }

  void openEditForm(Mahasiswa mhs) {
    editingId = mhs.id;
    nimC.text = mhs.nim;
    namaC.text = mhs.nama;
    semesterC.text = mhs.semester.toString();
    selectedProdi.value = mhs.prodi;
    Get.toNamed(AppRoutes.form);
  }

  // ------------------ CREATE & UPDATE ------------------
  Future<void> save() async {
    if (!formKey.currentState!.validate()) return;

    final nim = nimC.text.trim();

    // Cegah NIM ganda
    final nimSudahAda =
        mahasiswaList.any((m) => m.nim == nim && m.id != editingId);
    if (nimSudahAda) {
      Get.snackbar('Gagal', 'NIM $nim sudah terdaftar',
          snackPosition: SnackPosition.BOTTOM);
      return;
    }

    final mhs = Mahasiswa(
      id: editingId ?? DateTime.now().millisecondsSinceEpoch.toString(),
      nim: nim,
      nama: namaC.text.trim(),
      prodi: selectedProdi.value,
      semester: int.parse(semesterC.text.trim()),
    );

    if (isEdit) {
      await _service.update(mhs);
```

```
    } else {
      await _service.create(mhs);
    }

    final pesan = isEdit ? 'Data berhasil diperbarui' : 'Data berhasil ditambahkan';
    loadData();
    tutup(); // kembali ke halaman daftar
    Get.snackbar('Sukses', pesan, snackPosition: SnackPosition.BOTTOM);
  }

  // ------------------ DELETE ------------------
  Future<void> confirmDelete(Mahasiswa mhs) async {
    final yakin = await Get.dialog<bool>(
      AlertDialog(
        title: const Text('Hapus Data'),
        content: Text('Yakin ingin menghapus ${mhs.nama}?'),
        actions: [
          TextButton(
            onPressed: () => tutup(false),
            child: const Text('Batal'),
          ),
          FilledButton(
            onPressed: () => tutup(true),
            child: const Text('Hapus'),
          ),
        ],
      ),
    );

    if (yakin == true) {
      await _service.delete(mhs.id);
      loadData();
      Get.snackbar('Terhapus', '${mhs.nama} telah dihapus',
          snackPosition: SnackPosition.BOTTOM);
    }
  }

  // ------------------ HELPER NAVIGASI ------------------
  /// Menutup halaman / dialog paling atas (opsional membawa [result]).
  ///
  /// Kenapa tidak memakai Get.back()? Pada GetX 4.x, jika masih ada
  /// snackbar yang tampil, Get.back() HANYA menutup snackbar tersebut,
  /// sehingga halaman form / dialog tidak tertutup. Get.key adalah
  /// navigator milik GetMaterialApp, jadi pop() selalu menutup route.
  void tutup<T>([T? result]) => Get.key.currentState?.pop<T>(result);

  // ------------------ VALIDATOR ------------------
  String? validateNim(String? v) {
    if (v == null || v.trim().isEmpty) return 'NIM wajib diisi';
    if (int.tryParse(v.trim()) == null) return 'NIM harus berupa angka';
    if (v.trim().length < 8) return 'NIM minimal 8 digit';
    return null;
  }

  String? validateNama(String? v) {
    if (v == null || v.trim().isEmpty) return 'Nama wajib diisi';
    return null;
  }
```

```
  String? validateSemester(String? v) {
    final s = int.tryParse(v?.trim() ?? '');
    if (s == null) return 'Semester wajib berupa angka';
    if (s < 1 || s > 14) return 'Semester antara 1 - 14';
    return null;
  }

  @override
  void onClose() {
    nimC.dispose();
    namaC.dispose();
    semesterC.dispose();
    super.onClose();
  }
}
```

Bagian 1: State reaktif

```
final mahasiswaList = <Mahasiswa>[].obs;   // RxList<Mahasiswa>
final keyword = ''.obs;                     // RxString
final selectedProdi = 'Informatika'.obs;    // RxString
```

Akhiran `.obs` mengubah nilai biasa menjadi nilai yang bisa "diamati". Setiap `Obx` di View yang membaca nilai ini akan otomatis dibangun ulang saat nilainya berubah. Ingat cara membaca dan mengubahnya:

- `keyword.value = 'budi'` untuk `RxString` / `RxInt` (pakai `.value`).
- `mahasiswaList.assignAll(data)` untuk `RxList` (mengganti seluruh isi list sekaligus dan memicu satu kali rebuild).

`daftarProdi` tidak diberi `.obs` karena isinya tidak pernah berubah.

Bagian 2: State form

`formKey` dan tiga `TextEditingController` disimpan di controller (bukan di View) agar isi form bisa diisi dari controller saat mode edit, dan dibaca saat `save()`. Variabel `editingId` menentukan mode form:

`editingId`	`isEdit`	Judul form	Tombol	Saat disimpan
`null`	`false`	Tambah Mahasiswa	Simpan	id baru dibuat, `_service.create()`
berisi id	`true`	Edit Mahasiswa	Perbarui	id lama dipakai, `_service.update()`

Dengan cara ini **satu halaman form** dipakai untuk tambah sekaligus edit.

Bagian 3: `filteredList` (pencarian)

Getter ini mengembalikan seluruh data jika kata kunci kosong, atau hanya data yang nama (huruf kecil) atau NIM-nya mengandung kata kunci. Karena getter ini membaca `keyword.value` dan `mahasiswaList`, `Obx` di View yang memanggilnya akan rebuild setiap kali pengguna mengetik **atau** data berubah. Pencarian dilakukan di memori, jadi hasilnya instan.

Bagian 4: Siklus hidup `onInit` dan `onClose`

- `onInit()` dijalankan sekali saat controller pertama kali dibuat. Di sini kita memanggil `loadData()` sehingga daftar langsung terisi saat aplikasi dibuka.
- `onClose()` dijalankan saat controller dihapus dari memori. Di sini `TextEditingController` di-`dispose()` agar tidak bocor memori (sama seperti `dispose()` pada `StatefulWidget`).

Bagian 5: READ dengan `loadData()`

Mengambil semua data dari `HiveService`, mengurutkannya berdasarkan nama tanpa membedakan huruf besar/kecil, lalu memasukkannya ke `mahasiswaList`. Tanda `..` (cascade) berarti "panggil `sort` pada list yang sama, lalu kembalikan list-nya". Method ini dipanggil ulang setiap kali data berubah, sehingga `mahasiswaList` selalu sama dengan isi box.

Bagian 6: Membuka form

- `openAddForm()` mengosongkan `editingId` dan semua field, mengembalikan prodi ke pilihan pertama, lalu pindah ke route `/form`.
- `openEditForm(mhs)` mengisi `editingId` dan semua field dengan data mahasiswa yang dipilih (semester diubah ke teks dengan `toString()`), lalu pindah ke `/form`.

Karena isi field disiapkan **sebelum** pindah halaman, form langsung tampil terisi saat dibuka.

Bagian 7: CREATE & UPDATE dengan `save()`

1. `formKey.currentState!.validate()` menjalankan semua validator di form. Jika ada yang gagal, pesan merah muncul di bawah field dan fungsi berhenti.
2. **Cek NIM ganda**: `any(...)` mencari mahasiswa lain dengan NIM yang sama. Syarat `m.id != editingId` penting: saat mengedit, data milik sendiri tidak dihitung sebagai duplikat.
3. Membuat objek `Mahasiswa`. Saat tambah, id dibuat dari `DateTime.now().millisecondsSinceEpoch` (jumlah milidetik sejak 1970), cukup unik untuk satu perangkat. Saat edit, id lama dipakai agar `put` menimpa data yang sama.
4. `await` create atau update, lalu `loadData()` agar daftar ikut berubah.
5. `tutup()` kembali ke halaman daftar, lalu `Get.snackbar` menampilkan pesan sukses.

Perhatikan bahwa variabel `pesan` dihitung sebelum menutup form. Ini kebiasaan baik: ambil semua nilai yang dibutuhkan dulu, baru pindah halaman.

Bagian 8: DELETE dengan `confirmDelete()`

`Get.dialog<bool>(...)` menampilkan dialog dan **menunggu** sampai dialog ditutup. Tombol *Batal* menutup dengan nilai `false`, tombol *Hapus* dengan nilai `true`. Jika pengguna mengetuk area di luar dialog, hasilnya `null`. Karena itu pengecekannya `yakin == true`, bukan sekadar `if (yakin)`. Setelah yakin, data dihapus, daftar dimuat ulang, dan snackbar ditampilkan.

Bagian 9: Helper `tutup()`

```
void tutup<T>([T? result]) => Get.key.currentState?.pop<T>(result);
```

Kenapa tidak memakai `Get.back()` seperti biasa? Pada GetX 4.x, jika masih ada snackbar yang tampil (misalnya snackbar "NIM sudah terdaftar"), `Get.back()` hanya menutup snackbar tersebut dan halaman form tetap terbuka. Pengguna merasa tombolnya tidak bekerja. `Get.key` adalah navigator milik `GetMaterialApp`, sehingga `pop()` selalu menutup halaman atau dialog paling atas. Parameter `result` dipakai dialog hapus untuk mengirim `true` / `false`.

Bagian 10: Validator

Validator	Aturan	Pesan error
`validateNim`	Tidak kosong, hanya angka, minimal 8 digit	NIM wajib diisi / NIM harus berupa angka / NIM minimal 8 digit
`validateNama`	Tidak kosong (spasi saja dianggap kosong)	Nama wajib diisi
`validateSemester`	Berupa angka 1 sampai 14	Semester wajib berupa angka / Semester antara 1 - 14

Aturannya sama dengan Modul 05: validator mengembalikan `null` jika valid, atau teks pesan error jika tidak valid. `trim()` membuang spasi di awal dan akhir sehingga input " " tidak lolos.

ℹ️ **Controller ini mengimpor `material.dart`**

Di Modul 08 dan 11 dianjurkan agar controller tidak mengimpor `material.dart` supaya mudah di-unit-test. Proyek ini sengaja mengambil jalan yang lebih praktis untuk pemula: `GlobalKey<FormState>`, `TextEditingController`, dan `AlertDialog` ditaruh di controller sehingga View menjadi sangat tipis dan semua logika ada di satu tempat. Untuk proyek besar, Anda bisa memindahkan bagian form dan dialog ke View, lalu controller hanya menerima data bersih seperti `simpan(nim, nama, prodi, semester)`. Keduanya tetap MVC; yang berbeda hanya seberapa tipis View-nya.

7. Routes & Binding

Sebelum membuat View, kita siapkan dua hal dari GetX: **Routes** (alamat setiap halaman) dan **Binding** (siapa yang membuat controller). Dengan keduanya, View tidak perlu membuat controller sendiri dan navigasi cukup memakai nama, misalnya `Get.toNamed('/form')`.

1. Nama route

Buat file `lib/routes/app_routes.dart`:

`lib/routes/app_routes.dart`

```dart
/// Daftar nama route aplikasi.
abstract class AppRoutes {
  static const home = '/';
  static const form = '/form';
}
```

Nama route ditulis sebagai konstanta agar tidak salah ketik. Menulis `Get.toNamed(AppRoutes.form)` akan dicek oleh compiler, sedangkan `Get.toNamed('/from')` (salah ketik) baru ketahuan saat aplikasi berjalan. `abstract class` dipakai agar class ini tidak bisa dibuat objeknya; ia hanya wadah konstanta.

2. Binding

Buat file `lib/bindings/mahasiswa_binding.dart`:

`lib/bindings/mahasiswa_binding.dart`

```dart
import 'package:get/get.dart';

import '../controllers/mahasiswa_controller.dart';

/// Binding: mendaftarkan dependency (controller) ke GetX
/// sebelum halaman dibuka.
class MahasiswaBinding extends Bindings {
  @override
  void dependencies() {
    Get.lazyPut<MahasiswaController>(() => MahasiswaController());
  }
}
```

- `dependencies()` dipanggil GetX tepat sebelum halaman yang memakai binding ini dibuka.
- `Get.lazyPut` mendaftarkan "resep" pembuat controller. Controller baru benar-benar dibuat saat pertama kali dicari dengan `Get.find` (atau lewat properti `controller` milik `GetView`). Sesudah itu, objek yang **sama** dipakai di mana pun.

3. Pemetaan route ke halaman

Buat file `lib/routes/app_pages.dart`:

lib/routes/app_pages.dart

```dart
import 'package:get/get.dart';

import '../bindings/mahasiswa_binding.dart';
import '../views/mahasiswa_form_view.dart';
import '../views/mahasiswa_list_view.dart';
import 'app_routes.dart';

/// Pemetaan route -> halaman (+ binding).
class AppPages {
  static final pages = [
    GetPage(
      name: AppRoutes.home,
      page: () => const MahasiswaListView(),
      binding: MahasiswaBinding(),
    ),
    GetPage(
      name: AppRoutes.form,
      page: () => const MahasiswaFormView(),
    ),
  ];
}
```

Setiap `GetPage` menghubungkan nama route dengan widget halaman. Hal penting yang perlu diperhatikan:

- Binding hanya dipasang di route `home`. Halaman form **tidak** memerlukan binding sendiri karena selalu dibuka dari halaman daftar, sehingga `MahasiswaController` sudah ada di memori. Halaman form memakai controller yang **sama** dengan halaman daftar, dan inilah yang membuat data form (yang diisi di `openEditForm`) langsung terbaca di halaman form.
- Controller milik route `home` tetap hidup selama halaman daftar masih ada di tumpukan navigasi. Karena halaman daftar adalah halaman pertama, controller hidup sepanjang aplikasi berjalan.

> ⚠️ **Garis merah di `app_pages.dart`**
>
> File ini mengimpor `mahasiswa_list_view.dart` dan `mahasiswa_form_view.dart` yang belum dibuat, sehingga editor menampilkan error. Ini normal. Error akan hilang setelah kedua View dibuat di section 8 dan 9.

8. View Daftar Mahasiswa (Read, Cari, Hapus)

Sekarang lapisan paling atas: **View**. Halaman pertama adalah daftar mahasiswa. Halaman ini menangani **Read** (menampilkan data), pencarian, serta tombol menuju **Create**, **Update**, dan **Delete**.

Buat file `lib/views/mahasiswa_list_view.dart`:

`lib/views/mahasiswa_list_view.dart`

```dart
import 'package:flutter/material.dart';
import 'package:get/get.dart';

import '../controllers/mahasiswa_controller.dart';

/// VIEW: halaman daftar mahasiswa (READ + tombol Edit/Hapus).
/// `GetView<T>` otomatis menyediakan properti `controller`.
class MahasiswaListView extends GetView<MahasiswaController> {
  const MahasiswaListView({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text('Data Mahasiswa'),
        centerTitle: true,
      ),
      body: Column(
        children: [
          // ---------- Kolom pencarian ----------
          Padding(
            padding: const EdgeInsets.all(12),
            child: TextField(
              onChanged: (v) => controller.keyword.value = v,
              decoration: const InputDecoration(
                hintText: 'Cari nama atau NIM...',
                prefixIcon: Icon(Icons.search),
                border: OutlineInputBorder(),
              ),
            ),
          ),

          // ---------- Daftar data (reaktif) ----------
          Expanded(
            child: Obx(() {
              final data = controller.filteredList;

              if (data.isEmpty) {
                return const Center(
                  child: Text('Belum ada data mahasiswa'),
                );
              }

              return ListView.builder(
                padding: const EdgeInsets.only(bottom: 80),
                itemCount: data.length,
```

```
                itemBuilder: (context, index) {
                  final mhs = data[index];
                  return Card(
                    margin: const EdgeInsets.symmetric(
                        horizontal: 12, vertical: 6),
                    child: ListTile(
                      leading: CircleAvatar(
                        child: Text(mhs.nama[0].toUpperCase()),
                      ),
                      title: Text(mhs.nama),
                      subtitle: Text(
                          'NIM ${mhs.nim}\n${mhs.prodi} • Smt ${mhs.semester}'),
                      isThreeLine: true,
                      trailing: Row(
                        mainAxisSize: MainAxisSize.min,
                        children: [
                          IconButton(
                            visualDensity: VisualDensity.compact,
                            icon: const Icon(Icons.edit, color: Colors.orange),
                            onPressed: () => controller.openEditForm(mhs),
                          ),
                          IconButton(
                            visualDensity: VisualDensity.compact,
                            icon: const Icon(Icons.delete, color: Colors.red),
                            onPressed: () => controller.confirmDelete(mhs),
                          ),
                        ],
                      ),
                    ),
                  );
                },
              );
            }),
          ),
        ],
      ),
      floatingActionButton: FloatingActionButton.extended(
        onPressed: controller.openAddForm,
        icon: const Icon(Icons.add),
        label: const Text('Tambah'),
      ),
    );
  }
}
```

Penjelasan kode

- `GetView<MahasiswaController>` adalah `StatelessWidget` yang sudah punya properti `controller`. Di belakang layar properti ini memanggil `Get.find<MahasiswaController>()`, yaitu controller yang didaftarkan `MahasiswaBinding`. Kita tidak memerlukan `StatefulWidget` sama sekali.
- **Kolom pencarian**: setiap huruf yang diketik dikirim ke `controller.keyword.value`. Kolom ini tidak perlu dibungkus `Obx` karena ia tidak menampilkan state apa pun.
- `Obx(() { ... })` hanya membungkus bagian yang berubah, yaitu daftar data. Di dalamnya `controller.filteredList` dibaca, sehingga `Obx` otomatis "berlangganan" ke `keyword` dan

`mahasiswaList`. AppBar, kolom cari, dan tombol Tambah tidak ikut dibangun ulang.

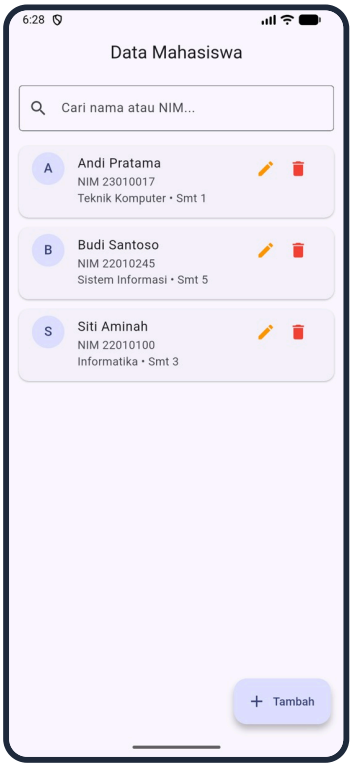
- **Empty state**: jika daftar kosong (belum ada data atau pencarian tidak menemukan hasil), tampil teks *Belum ada data mahasiswa* di tengah layar.
- `ListView.builder` membuat kartu hanya untuk item yang terlihat di layar (Modul 06). `padding bottom: 80` memberi ruang agar kartu terakhir tidak tertutup tombol Tambah.
- **Isi kartu**: `CircleAvatar` berisi huruf pertama nama, `subtitle` dua baris (NIM, lalu prodi dan semester) dengan `isThreeLine: true` agar tinggi kartu cukup.
- **Tombol aksi**: ikon pensil memanggil `controller.openEditForm(mhs)`, ikon tempat sampah memanggil `controller.confirmDelete(mhs)`, dan tombol *Tambah* memanggil `controller.openAddForm`. View hanya meneruskan klik; semua keputusan ada di controller.

> 💡 `onPressed: controller.openAddForm` **vs** `onPressed: () => controller.openEditForm(mhs)`
>
> Jika method tidak butuh argumen, kita bisa langsung memberikan method-nya (*tear-off*) tanpa tanda kurung. Jika butuh argumen seperti `mhs`, bungkus dengan fungsi anonim `() => ...`. Kesalahan umum adalah menulis `onPressed: controller.openAddForm()` (dengan kurung), yang membuat method langsung dijalankan saat build, bukan saat tombol ditekan.

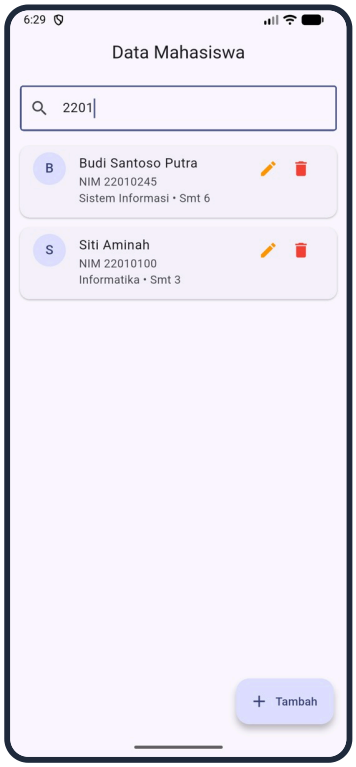
Hasilnya saat aplikasi dijalankan (setelah section 10 selesai):



Pertama kali dibuka: box masih kosong



Setelah tiga data ditambahkan, terurut A-Z



Mengetik `2201`: hanya NIM yang cocok

9. View Form (Create & Update)

Halaman kedua adalah form yang dipakai untuk **Create** sekaligus **Update**. Mode form ditentukan oleh `controller.isEdit` yang sudah diatur sebelum halaman dibuka.

Buat file `lib/views/mahasiswa_form_view.dart` :

`lib/views/mahasiswa_form_view.dart`

```dart
import 'package:flutter/material.dart';
import 'package:flutter/services.dart';
import 'package:get/get.dart';

import '../controllers/mahasiswa_controller.dart';

/// VIEW: halaman form untuk CREATE dan UPDATE.
class MahasiswaFormView extends GetView<MahasiswaController> {
  const MahasiswaFormView({super.key});

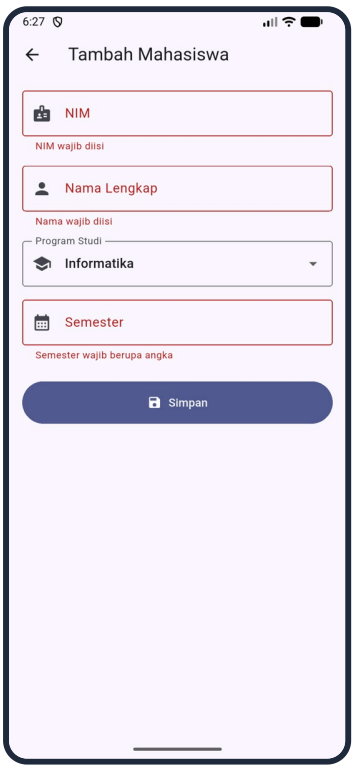
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: Text(controller.isEdit ? 'Edit Mahasiswa' : 'Tambah Mahasiswa'),
      ),
      body: Form(
        key: controller.formKey,
        // pesan error diperbarui otomatis saat user mengetik
        autovalidateMode: AutovalidateMode.onUserInteraction,
        child: ListView(
          padding: const EdgeInsets.all(16),
          children: [
            TextFormField(
              controller: controller.nimC,
              keyboardType: TextInputType.number,
              inputFormatters: [FilteringTextInputFormatter.digitsOnly],
              validator: controller.validateNim,
              decoration: const InputDecoration(
                labelText: 'NIM',
                prefixIcon: Icon(Icons.badge),
                border: OutlineInputBorder(),
              ),
            ),
            const SizedBox(height: 16),
            TextFormField(
              controller: controller.namaC,
              textCapitalization: TextCapitalization.words,
              validator: controller.validateNama,
              decoration: const InputDecoration(
                labelText: 'Nama Lengkap',
                prefixIcon: Icon(Icons.person),
                border: OutlineInputBorder(),
              ),
            ),
            const SizedBox(height: 16),
            DropdownButtonFormField<String>(
              initialValue: controller.selectedProdi.value,
```

```
                items: controller.daftarProdi
                    .map((p) => DropdownMenuItem(value: p, child: Text(p)))
                    .toList(),
                onChanged: (v) {
                  if (v != null) controller.selectedProdi.value = v;
                },
                decoration: const InputDecoration(
                  labelText: 'Program Studi',
                  prefixIcon: Icon(Icons.school),
                  border: OutlineInputBorder(),
                ),
              ),
              const SizedBox(height: 16),
              TextFormField(
                controller: controller.semesterC,
                keyboardType: TextInputType.number,
                inputFormatters: [FilteringTextInputFormatter.digitsOnly],
                validator: controller.validateSemester,
                decoration: const InputDecoration(
                  labelText: 'Semester',
                  prefixIcon: Icon(Icons.calendar_month),
                  border: OutlineInputBorder(),
                ),
              ),
              const SizedBox(height: 24),
              FilledButton.icon(
                onPressed: controller.save,
                icon: const Icon(Icons.save),
                label: Text(controller.isEdit ? 'Perbarui' : 'Simpan'),
                style: FilledButton.styleFrom(
                  padding: const EdgeInsets.symmetric(vertical: 16),
                ),
              ),
            ],
          ),
        ),
      ),
    );
  }
}
```

Penjelasan kode

- **Judul dan label tombol dinamis**: `controller.isEdit ? 'Edit Mahasiswa' : 'Tambah Mahasiswa'`. Tidak perlu `Obx` karena nilainya sudah pasti saat halaman dibangun dan tidak berubah selama form terbuka.
- **`Form(key: controller.formKey)`** mengelompokkan semua field sehingga `validate()` di controller bisa memeriksa semuanya sekaligus.
- **`autovalidateMode: AutovalidateMode.onUserInteraction`**: setelah pengguna menyentuh sebuah field, pesan error diperbarui setiap kali ia mengetik. Pesan merah langsung hilang begitu input sudah benar.
- **`controller: controller.nimC`** : isi field disimpan di `TextEditingController` milik controller. Inilah yang membuat form mode edit langsung terisi.

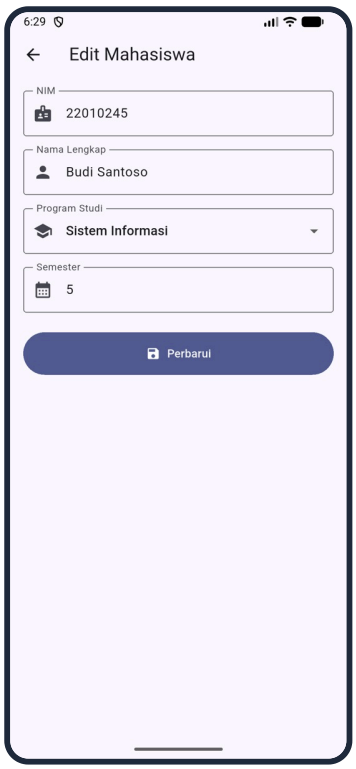
- `keyboardType: TextInputType.number` menampilkan keyboard angka, dan `FilteringTextInputFormatter.digitsOnly` (dari `package:flutter/services.dart`) menolak karakter selain angka, misalnya jika pengguna menempel teks.
- `textCapitalization: TextCapitalization.words` membuat huruf pertama setiap kata otomatis kapital di keyboard.
- `validator: controller.validateNim`: fungsi validasi diambil dari controller (tear-off), sehingga aturan validasi ada di satu tempat.
- `DropdownButtonFormField`: `initialValue` diambil dari `selectedProdi` (saat edit berisi prodi lama). Saat pengguna memilih, `onChanged` menulis ke `controller.selectedProdi.value`, yang nanti dibaca `save()`.
- `FilledButton.icon(onPressed: controller.save)` menjalankan validasi lalu simpan.



Simpan ditekan saat form kosong



Memilih program studi



Mode edit: field terisi, tombol berubah menjadi Perbarui

ℹ️ `initialValue` pada dropdown

Di Flutter versi terbaru, parameter `value` pada `DropdownButtonFormField` sudah ditandai *deprecated* dan diganti `initialValue`. Jika Flutter di laptop Anda versi lama dan muncul error *No named parameter with the name 'initialValue'*, jalankan `flutter upgrade`, atau untuk sementara ganti `initialValue:` menjadi `value:`.

10. main.dart: Inisialisasi Hive & GetMaterialApp

Semua bagian sudah siap. File terakhir, `main.dart`, menyalakan Hive lalu menjalankan aplikasi dengan `GetMaterialApp`.

Hapus **seluruh** isi `lib/main.dart` bawaan, lalu ganti dengan:

`lib/main.dart`

```dart
import 'package:flutter/material.dart';
import 'package:get/get.dart';
import 'package:hive_ce_flutter/hive_ce_flutter.dart';

import 'models/mahasiswa_adapter.dart';
import 'models/mahasiswa_model.dart';
import 'routes/app_pages.dart';
import 'routes/app_routes.dart';
import 'services/hive_service.dart';

Future<void> main() async {
  // 1. Pastikan binding Flutter siap sebelum memanggil kode async
  WidgetsFlutterBinding.ensureInitialized();

  // 2. Inisialisasi Hive (menentukan folder penyimpanan otomatis)
  await Hive.initFlutter();

  // 3. Daftarkan adapter untuk class Mahasiswa
  Hive.registerAdapter(MahasiswaAdapter());

  // 4. Buka box (seperti membuka "tabel")
  await Hive.openBox<Mahasiswa>(HiveService.boxName);

  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return GetMaterialApp(
      title: 'CRUD Mahasiswa',
      debugShowCheckedModeBanner: false,
      theme: ThemeData(
        colorSchemeSeed: Colors.indigo,
        useMaterial3: true,
      ),
      initialRoute: AppRoutes.home,
      getPages: AppPages.pages,
    );
  }
}
```

Empat langkah wajib sebelum `runApp`

1. `WidgetsFlutterBinding.ensureInitialized()` : `main()` kita bersifat `async` dan memanggil plugin (Hive butuh lokasi folder dari sistem operasi). Baris ini memastikan "jembatan" Flutter ke sistem operasi sudah siap sebelum plugin dipakai.
2. `await Hive.initFlutter()` : menentukan folder penyimpanan file box di HP secara otomatis.
3. `Hive.registerAdapter(MahasiswaAdapter())` : memperkenalkan adapter ke Hive. Harus dilakukan **sebelum** membuka box yang berisi `Mahasiswa`, karena saat dibuka Hive langsung membaca isi file dan butuh adapter untuk menerjemahkannya.
4. `await Hive.openBox<Mahasiswa>(HiveService.boxName)` : membuka box (mirip membuka tabel). Setelah ini, `Hive.box()` di `HiveService` bisa dipakai secara sinkron di mana saja.

Jika urutan ini salah, Anda akan melihat error seperti di section 14.

`GetMaterialApp`

- `GetMaterialApp` menggantikan `MaterialApp` agar fitur GetX (named route, `Get.snackbar`, `Get.dialog`, `Get.key`) bisa dipakai tanpa `BuildContext`.
- `initialRoute: AppRoutes.home` membuka halaman daftar pertama kali, dan saat itu `MahasiswaBinding` dijalankan sehingga `MahasiswaController` dibuat dan `onInit()` memuat data dari box.
- `getPages: AppPages.pages` adalah daftar semua route dari section 7.
- `colorSchemeSeed: Colors.indigo` membuat seluruh warna tema Material 3 dari satu warna dasar.

Cek sebelum menjalankan

```
flutter analyze
```

Hasil yang diharapkan: `No issues found!`. Jika masih ada error, biasanya karena salah ketik nama file atau nama class. Bandingkan dengan listing lengkap di section 13.

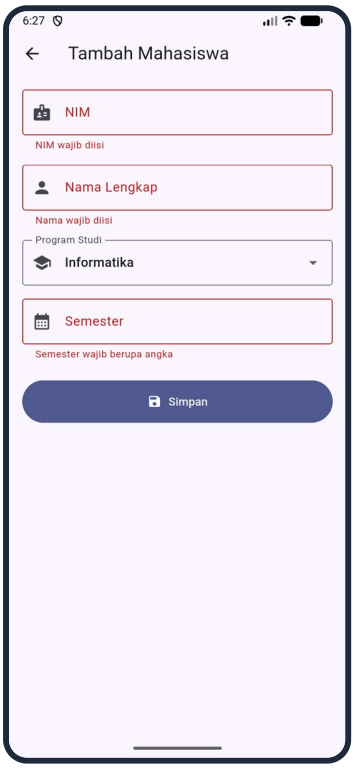
11. Menjalankan & Menguji Alur CRUD

Saatnya menjalankan aplikasi dan menguji keempat operasi CRUD satu per satu. Ikuti skenario di bawah ini dan cocokkan hasilnya dengan screenshot.

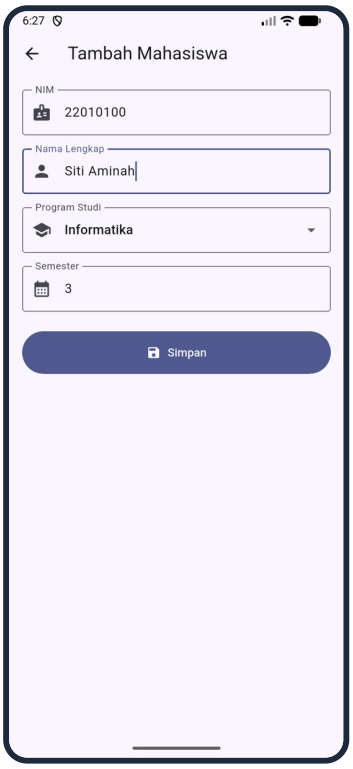
```
flutter run
```

Skenario 1: Validasi dan Create

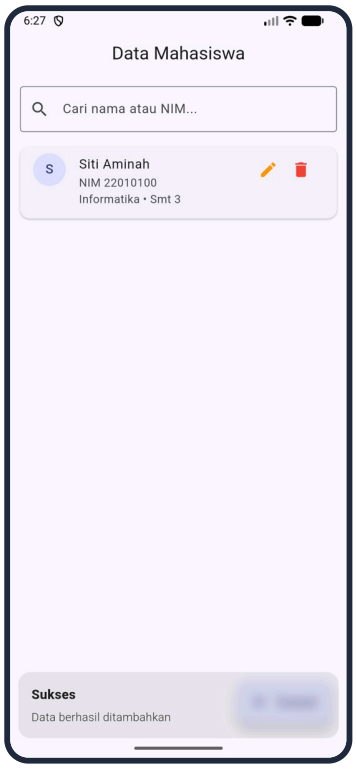
1. Tekan *Tambah*, lalu langsung tekan *Simpan* tanpa mengisi apa pun. Tiga pesan merah harus muncul.
2. Isi NIM `22010100`, nama `Siti Aminah`, prodi *Informatika*, semester `3`, lalu *Simpan*.
3. Aplikasi kembali ke daftar dan snackbar *Sukses: Data berhasil ditambahkan* muncul.



Langkah 1: validasi berjalan



Langkah 2: form terisi benar



Langkah 3: data tampil di daftar

Tambahkan dua data lagi: `22010245` Budi Santoso (Sistem Informasi, semester 5) dan `23010017` Andi Pratama (Teknik Komputer, semester 1). Daftar akan terurut berdasarkan nama: Andi, Budi, Siti.

Skenario 2: NIM ganda ditolak

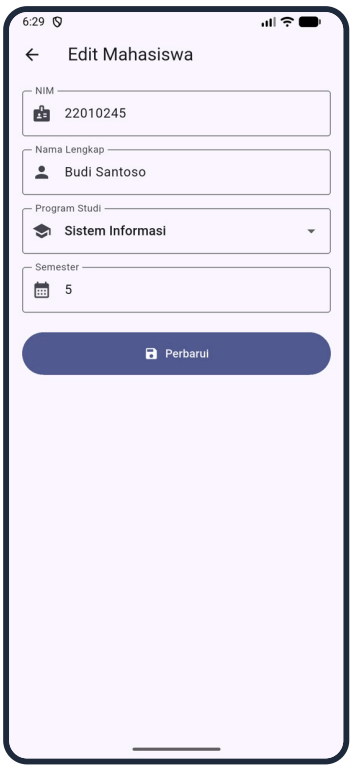
Tambah data baru dengan NIM `22010100` (milik Siti). Saat *Simpan*, form tidak tertutup dan muncul snackbar *Gagal: NIM 22010100 sudah terdaftar*.

Skenario 3: Update

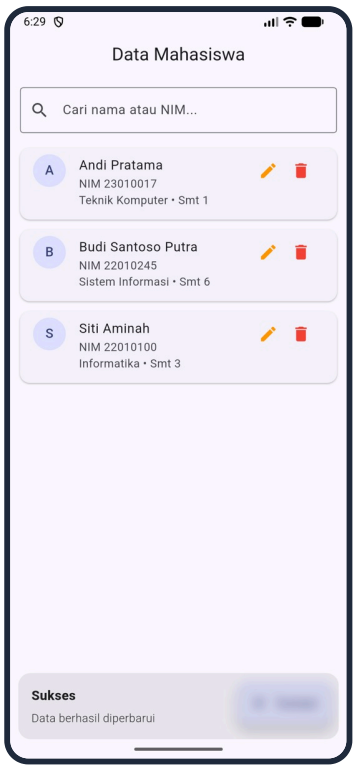
Tekan ikon pensil di kartu Budi. Form terbuka dengan judul *Edit Mahasiswa* dan data lama sudah terisi. Ubah nama menjadi `Budi Santoso Putra` dan semester menjadi `6`, lalu *Perbarui*.



Skenario 2: NIM ganda ditolak



Skenario 3: form edit terisi data lama



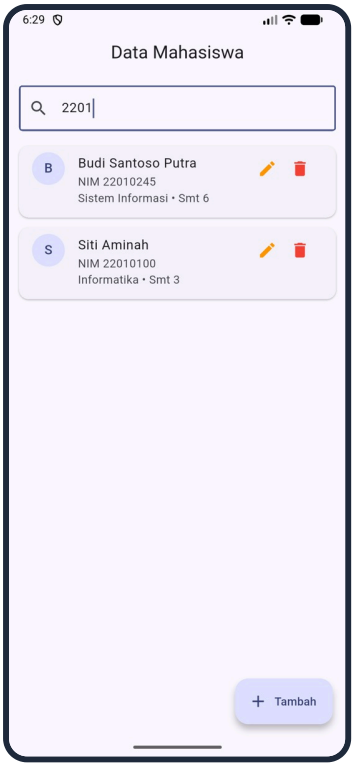
Skenario 3: data Budi berubah

Skenario 4: Cari

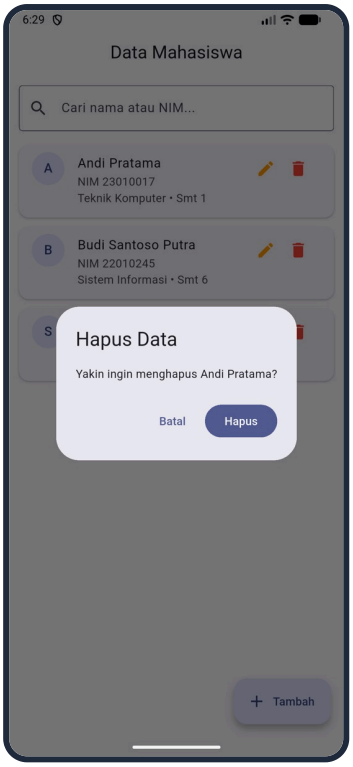
Ketik `2201` di kolom pencarian. Hanya Budi dan Siti yang tampil karena NIM keduanya mengandung `2201`. Coba juga ketik `siti` (huruf kecil): pencarian nama tidak membedakan huruf besar/kecil. Hapus isi kolom pencarian untuk menampilkan semua data lagi.

Skenario 5: Delete dan bukti data tersimpan

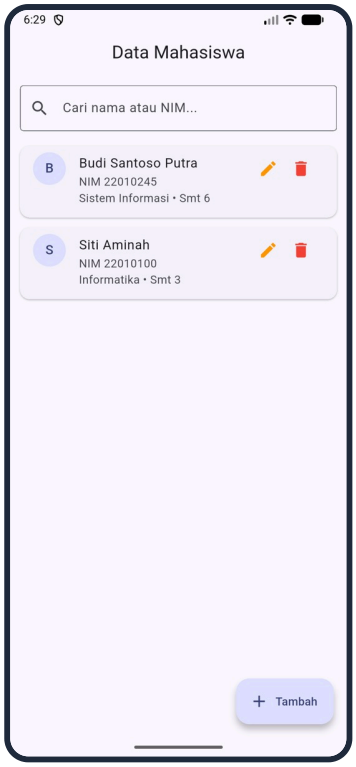
1. Tekan ikon tempat sampah pada kartu Andi. Dialog konfirmasi muncul. Tekan *Batal* dulu dan pastikan data tidak terhapus.
2. Ulangi, lalu tekan *Hapus*. Andi hilang dari daftar dan snackbar *Terhapus* muncul.
3. **Tutup paksa aplikasi** (geser dari daftar aplikasi terbaru, atau tekan q di terminal `flutter run`), lalu buka lagi. Data Budi dan Siti harus tetap ada. Inilah bukti data tersimpan di Hive, bukan hanya di memori.



Skenario 4: hasil pencarian



Skenario 5: konfirmasi hapus



Skenario 5: setelah ditutup paksa dan dibuka lagi

> 💡 **Ingin mengulang dari data kosong?**
>
> Uninstall aplikasi dari emulator (tekan lama ikon aplikasi, lalu *Uninstall*) atau hapus data aplikasi lewat *Settings > Apps > crud_mahasiswa > Storage > Clear storage*. File box ikut terhapus.

12. Unit Test & Widget Test

Menguji manual seperti section 11 memakan waktu dan mudah terlewat. Dua file test berikut menjalankan pengecekan yang sama secara otomatis dalam beberapa detik: satu **unit test** untuk `HiveService` dan satu **widget test** untuk alur CRUD lewat UI.

1. Unit test `HiveService`

Buat file `test/hive_service_test.dart`:

`test/hive_service_test.dart`

```dart
import 'dart:io';

import 'package:crud_mahasiswa/models/mahasiswa_adapter.dart';
import 'package:crud_mahasiswa/models/mahasiswa_model.dart';
import 'package:crud_mahasiswa/services/hive_service.dart';
import 'package:flutter_test/flutter_test.dart';
import 'package:hive_ce/hive_ce.dart';

void main() {
  late Directory dir;
  final service = HiveService();

  setUpAll(() async {
    dir = await Directory.systemTemp.createTemp('hive_test');
    Hive.init(dir.path);
    Hive.registerAdapter(MahasiswaAdapter());
    await Hive.openBox<Mahasiswa>(HiveService.boxName);
  });

  tearDownAll(() async {
    await Hive.deleteFromDisk();
    await dir.delete(recursive: true);
  });

  final budi = Mahasiswa(
      id: '1', nim: '220101001', nama: 'Budi', prodi: 'Informatika', semester: 5);

  test('CREATE & READ', () async {
    await service.create(budi);
    final all = service.getAll();
    expect(all.length, 1);
    expect(all.first.nama, 'Budi');
    expect(all.first.semester, 5);
  });

  test('UPDATE', () async {
    await service.update(budi.copyWith(nama: 'Budi Santoso', semester: 6));
    final all = service.getAll();
    expect(all.length, 1);
    expect(all.first.nama, 'Budi Santoso');
    expect(all.first.semester, 6);
  });

  test('Data tetap ada setelah box ditutup & dibuka lagi', () async {
```

```
    await Hive.box<Mahasiswa>(HiveService.boxName).close();
    await Hive.openBox<Mahasiswa>(HiveService.boxName);
    expect(service.getAll().first.nama, 'Budi Santoso');
  });

  test('DELETE', () async {
    await service.delete('1');
    expect(service.getAll(), isEmpty);
  });
}
```

- `Hive.init(dir.path)`, bukan `Hive.initFlutter()`. Saat test, tidak ada HP/emulator sehingga folder penyimpanan dibuat sendiri di folder sementara (`Directory.systemTemp.createTemp`). Data test tidak tercampur dengan data aplikasi.
- `setUpAll` berjalan sekali sebelum semua test: menyiapkan Hive, mendaftarkan adapter, dan membuka box.
- `tearDownAll` berjalan sekali setelah semua test: menghapus file box dan folder sementara.
- Keempat test berjalan **berurutan** dan saling melanjutkan: CREATE menambah Budi, UPDATE mengubahnya, test ketiga menutup lalu membuka ulang box untuk membuktikan data benar-benar tertulis di file (mirip aplikasi ditutup lalu dibuka), dan DELETE menghapusnya.

2. Widget test alur CRUD

Buat file `test/crud_flow_test.dart`:

`test/crud_flow_test.dart`

```
import 'dart:io';

import 'package:crud_mahasiswa/main.dart';
import 'package:crud_mahasiswa/models/mahasiswa_adapter.dart';
import 'package:crud_mahasiswa/models/mahasiswa_model.dart';
import 'package:crud_mahasiswa/services/hive_service.dart';
import 'package:flutter/material.dart';
import 'package:flutter_test/flutter_test.dart';
import 'package:get/get.dart';
import 'package:hive_ce/hive_ce.dart';

/// Tap tombol lalu beri waktu nyata agar operasi IO Hive selesai.
Future<void> tapDanTunggu(WidgetTester tester, Finder f) async {
  await tester.tap(f);
  for (var i = 0; i < 5; i++) {
    await tester.pump(const Duration(milliseconds: 300));
    await tester.runAsync(
        () => Future.delayed(const Duration(milliseconds: 100)));
  }
}

void main() {
  testWidgets('Alur CRUD lewat UI', (tester) async {
    await tester.runAsync(() async {
      final dir = await Directory.systemTemp.createTemp('hive_ui');
      Hive.init(dir.path);
      Hive.registerAdapter(MahasiswaAdapter());
```

```
      await Hive.openBox<Mahasiswa>(HiveService.boxName);
    });

    await tester.pumpWidget(const MyApp());
    await tester.pumpAndSettle();
    expect(find.text('Belum ada data mahasiswa'), findsOneWidget);

    // CREATE
    await tester.tap(find.text('Tambah'));
    await tester.pumpAndSettle();
    await tester.enterText(find.widgetWithText(TextFormField, 'NIM'), '22010100');
    await tester.enterText(find.widgetWithText(TextFormField, 'Nama Lengkap'), 'Siti');
    await tester.enterText(find.widgetWithText(TextFormField, 'Semester'), '3');
    await tapDanTunggu(tester, find.text('Simpan'));
    await tester.pumpAndSettle(const Duration(seconds: 4));
    expect(find.text('Siti'), findsOneWidget);

    // UPDATE
    await tester.tap(find.byIcon(Icons.edit));
    await tester.pumpAndSettle();
    await tester.enterText(find.widgetWithText(TextFormField, 'Nama Lengkap'), 'Siti Aminah');
    await tapDanTunggu(tester, find.text('Perbarui'));
    await tester.pumpAndSettle(const Duration(seconds: 4));
    expect(find.text('Siti Aminah'), findsOneWidget);

    // DELETE
    await tester.tap(find.byIcon(Icons.delete));
    await tester.pumpAndSettle();
    await tapDanTunggu(tester, find.text('Hapus'));
    await tester.pumpAndSettle(const Duration(seconds: 4));
    expect(find.text('Belum ada data mahasiswa'), findsOneWidget);
    Get.reset();
  });
}
```

- Test ini menjalankan `MyApp` sungguhan, lalu "menekan tombol" dan "mengetik" seperti pengguna: tambah Siti, ubah menjadi Siti Aminah, lalu hapus.
- **`tester.runAsync`**: widget test berjalan dengan jam palsu (*fake async*), sedangkan Hive membaca/menulis file sungguhan. Operasi file harus dibungkus `runAsync` agar benar-benar dijalankan. Helper `tapDanTunggu` memberi sedikit waktu nyata setelah tombol yang memicu penulisan ke Hive ditekan.
- **`pumpAndSettle(const Duration(seconds: 4))`** menunggu animasi pindah halaman dan snackbar selesai.
- **`find.widgetWithText(TextFormField, 'NIM')`** mencari field berdasarkan label-nya.
- **`Get.reset()`** di akhir membersihkan semua controller GetX agar tidak terbawa ke test lain.

3. Jalankan

```
flutter analyze
flutter test
```

Hasil yang diharapkan:

```
Analyzing crud_mahasiswa...
No issues found!

00:03 +5: All tests passed!
```

Angka `+5` berarti lima test lulus: empat dari `hive_service_test.dart` dan satu dari `crud_flow_test.dart`.

> ⚠️ **Test gagal dengan pesan tentang `widget_test.dart` ?**
>
> Berarti file test bawaan belum dihapus (section 2). File itu mencari tombol `+` dan angka `0` milik aplikasi counter yang sudah tidak ada. Hapus file tersebut lalu jalankan ulang.

13. Listing Lengkap (Siap Copy-Paste)

Section ini berisi **seluruh** file proyek dalam bentuk final, siap disalin. Gunakan untuk membandingkan jika aplikasi Anda error, atau untuk membuat ulang proyek dari nol dengan cepat. Semua kode di bawah sudah diverifikasi dengan `flutter analyze` (tanpa masalah), `flutter test` (5 test lulus), dan dijalankan di Android Emulator.

Langkah dari proyek kosong

```
flutter create crud_mahasiswa
cd crud_mahasiswa
flutter pub add get hive_ce hive_ce_flutter
rm test/widget_test.dart
mkdir -p lib/models lib/services lib/controllers lib/bindings lib/routes lib/views
```

Lalu buat 12 file di bawah ini (10 file di `lib/`, 2 file di `test/`), dan jalankan:

```
flutter analyze
flutter test
flutter run
```

No	File	Lapisan
1	`lib/main.dart`	Titik awal aplikasi
2	`lib/models/mahasiswa_model.dart`	Model
3	`lib/models/mahasiswa_adapter.dart`	Model
4	`lib/services/hive_service.dart`	Model (akses data)
5	`lib/controllers/mahasiswa_controller.dart`	Controller
6	`lib/bindings/mahasiswa_binding.dart`	GetX (DI)
7	`lib/routes/app_routes.dart`	GetX (route)
8	`lib/routes/app_pages.dart`	GetX (route)
9	`lib/views/mahasiswa_list_view.dart`	View
10	`lib/views/mahasiswa_form_view.dart`	View
11	`test/hive_service_test.dart`	Unit test
12	`test/crud_flow_test.dart`	Widget test

pubspec.yaml (bagian dependencies)

Bagian ini dibuat otomatis oleh `flutter pub add`. Nomor versi di laptop Anda boleh lebih baru.

```yaml
dependencies:
  flutter:
    sdk: flutter

  cupertino_icons: ^1.0.8
  get: ^4.7.3
  hive_ce: ^2.23.3
  hive_ce_flutter: ^2.5.0
```

1. main.dart

`lib/main.dart`

```dart
import 'package:flutter/material.dart';
import 'package:get/get.dart';
import 'package:hive_ce_flutter/hive_ce_flutter.dart';
```

```dart
import 'models/mahasiswa_adapter.dart';
import 'models/mahasiswa_model.dart';
import 'routes/app_pages.dart';
import 'routes/app_routes.dart';
import 'services/hive_service.dart';

Future<void> main() async {
  // 1. Pastikan binding Flutter siap sebelum memanggil kode async
  WidgetsFlutterBinding.ensureInitialized();

  // 2. Inisialisasi Hive (menentukan folder penyimpanan otomatis)
  await Hive.initFlutter();

  // 3. Daftarkan adapter untuk class Mahasiswa
  Hive.registerAdapter(MahasiswaAdapter());

  // 4. Buka box (seperti membuka "tabel")
  await Hive.openBox<Mahasiswa>(HiveService.boxName);

  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return GetMaterialApp(
      title: 'CRUD Mahasiswa',
      debugShowCheckedModeBanner: false,
      theme: ThemeData(
        colorSchemeSeed: Colors.indigo,
        useMaterial3: true,
      ),
      initialRoute: AppRoutes.home,
      getPages: AppPages.pages,
    );
  }
}
```

2. Model Mahasiswa

lib/models/mahasiswa_model.dart

```dart
/// MODEL: representasi data Mahasiswa.
/// Class ini murni berisi data (tidak tahu soal UI maupun GetX).
class Mahasiswa {
  final String id;
  final String nim;
  final String nama;
  final String prodi;
  final int semester;

  Mahasiswa({
    required this.id,
    required this.nim,
    required this.nama,
```

```
    required this.prodi,
    required this.semester,
  });

  /// Membuat salinan objek dengan sebagian field yang diubah.
  Mahasiswa copyWith({
    String? nim,
    String? nama,
    String? prodi,
    int? semester,
  }) {
    return Mahasiswa(
      id: id,
      nim: nim ?? this.nim,
      nama: nama ?? this.nama,
      prodi: prodi ?? this.prodi,
      semester: semester ?? this.semester,
    );
  }
}
```

3. TypeAdapter

`lib/models/mahasiswa_adapter.dart`

```dart
import 'package:hive_ce/hive_ce.dart';

import 'mahasiswa_model.dart';

/// TypeAdapter memberi tahu Hive cara menyimpan (write)
/// dan membaca (read) objek Mahasiswa dalam bentuk biner.
///
/// ATURAN PENTING:
/// 1. typeId harus unik per class (0 - 223).
/// 2. Urutan read HARUS sama persis dengan urutan write.
class MahasiswaAdapter extends TypeAdapter<Mahasiswa> {
  @override
  final int typeId = 0;

  @override
  Mahasiswa read(BinaryReader reader) {
    return Mahasiswa(
      id: reader.readString(),
      nim: reader.readString(),
      nama: reader.readString(),
      prodi: reader.readString(),
      semester: reader.readInt(),
    );
  }

  @override
  void write(BinaryWriter writer, Mahasiswa obj) {
    writer.writeString(obj.id);
    writer.writeString(obj.nim);
    writer.writeString(obj.nama);
    writer.writeString(obj.prodi);
    writer.writeInt(obj.semester);
```

```
  }
}
```

4. HiveService

lib/services/hive_service.dart

```dart
import 'package:hive_ce/hive_ce.dart';

import '../models/mahasiswa_model.dart';

/// Lapisan akses data (bagian dari MODEL pada MVC).
/// Semua operasi CRUD ke Hive dikumpulkan di sini,
/// sehingga Controller tidak perlu tahu detail database.
class HiveService {
  static const String boxName = 'mahasiswa_box';

  Box<Mahasiswa> get _box => Hive.box<Mahasiswa>(boxName);

  // READ
  List<Mahasiswa> getAll() => _box.values.toList();

  // CREATE (key = id mahasiswa)
  Future<void> create(Mahasiswa mhs) => _box.put(mhs.id, mhs);

  // UPDATE (put dengan key yang sama akan menimpa data lama)
  Future<void> update(Mahasiswa mhs) => _box.put(mhs.id, mhs);

  // DELETE
  Future<void> delete(String id) => _box.delete(id);
}
```

5. MahasiswaController

lib/controllers/mahasiswa_controller.dart

```dart
import 'package:flutter/material.dart';
import 'package:get/get.dart';

import '../models/mahasiswa_model.dart';
import '../routes/app_routes.dart';
import '../services/hive_service.dart';

/// CONTROLLER: penghubung antara View dan Model.
/// Menyimpan state (reaktif) dan berisi seluruh logika CRUD.
class MahasiswaController extends GetxController {
  final HiveService _service = HiveService();

  // ------------------ STATE REAKTIF ------------------
  final mahasiswaList = <Mahasiswa>[].obs;
  final keyword = ''.obs;
  final selectedProdi = 'Informatika'.obs;

  final List<String> daftarProdi = const [
    'Informatika',
    'Sistem Informasi',
```

```
    'Teknik Komputer',
    'Data Science',
  ];

  // ------------------ STATE FORM ------------------
  final formKey = GlobalKey<FormState>();
  final nimC = TextEditingController();
  final namaC = TextEditingController();
  final semesterC = TextEditingController();

  /// null  -> mode TAMBAH
  /// berisi id -> mode EDIT
  String? editingId;
  bool get isEdit => editingId != null;

  /// Data yang ditampilkan (sudah difilter pencarian).
  List<Mahasiswa> get filteredList {
    final q = keyword.value.toLowerCase();
    if (q.isEmpty) return mahasiswaList;
    return mahasiswaList
        .where((m) =>
            m.nama.toLowerCase().contains(q) || m.nim.contains(q))
        .toList();
  }

  @override
  void onInit() {
    super.onInit();
    loadData();
  }

  // ------------------ READ ------------------
  void loadData() {
    final data = _service.getAll()
      ..sort((a, b) => a.nama.toLowerCase().compareTo(b.nama.toLowerCase()));
    mahasiswaList.assignAll(data);
  }

  // ------------------ NAVIGASI KE FORM ------------------
  void openAddForm() {
    editingId = null;
    nimC.clear();
    namaC.clear();
    semesterC.clear();
    selectedProdi.value = daftarProdi.first;
    Get.toNamed(AppRoutes.form);
  }

  void openEditForm(Mahasiswa mhs) {
    editingId = mhs.id;
    nimC.text = mhs.nim;
    namaC.text = mhs.nama;
    semesterC.text = mhs.semester.toString();
    selectedProdi.value = mhs.prodi;
    Get.toNamed(AppRoutes.form);
  }

  // ------------------ CREATE & UPDATE ------------------
  Future<void> save() async {
```

```
    if (!formKey.currentState!.validate()) return;

    final nim = nimC.text.trim();

    // Cegah NIM ganda
    final nimSudahAda =
        mahasiswaList.any((m) => m.nim == nim && m.id != editingId);
    if (nimSudahAda) {
      Get.snackbar('Gagal', 'NIM $nim sudah terdaftar',
          snackPosition: SnackPosition.BOTTOM);
      return;
    }

    final mhs = Mahasiswa(
      id: editingId ?? DateTime.now().millisecondsSinceEpoch.toString(),
      nim: nim,
      nama: namaC.text.trim(),
      prodi: selectedProdi.value,
      semester: int.parse(semesterC.text.trim()),
    );

    if (isEdit) {
      await _service.update(mhs);
    } else {
      await _service.create(mhs);
    }

    final pesan = isEdit ? 'Data berhasil diperbarui' : 'Data berhasil ditambahkan';
    loadData();
    tutup(); // kembali ke halaman daftar
    Get.snackbar('Sukses', pesan, snackPosition: SnackPosition.BOTTOM);
  }

  // ------------------ DELETE ------------------
  Future<void> confirmDelete(Mahasiswa mhs) async {
    final yakin = await Get.dialog<bool>(
      AlertDialog(
        title: const Text('Hapus Data'),
        content: Text('Yakin ingin menghapus ${mhs.nama}?'),
        actions: [
          TextButton(
            onPressed: () => tutup(false),
            child: const Text('Batal'),
          ),
          FilledButton(
            onPressed: () => tutup(true),
            child: const Text('Hapus'),
          ),
        ],
      ),
    );

    if (yakin == true) {
      await _service.delete(mhs.id);
      loadData();
      Get.snackbar('Terhapus', '${mhs.nama} telah dihapus',
          snackPosition: SnackPosition.BOTTOM);
    }
  }
```

```dart
  // ------------------ HELPER NAVIGASI ------------------
  /// Menutup halaman / dialog paling atas (opsional membawa [result]).
  ///
  /// Kenapa tidak memakai Get.back()? Pada GetX 4.x, jika masih ada
  /// snackbar yang tampil, Get.back() HANYA menutup snackbar tersebut,
  /// sehingga halaman form / dialog tidak tertutup. Get.key adalah
  /// navigator milik GetMaterialApp, jadi pop() selalu menutup route.
  void tutup<T>([T? result]) => Get.key.currentState?.pop<T>(result);

  // ------------------ VALIDATOR ------------------
  String? validateNim(String? v) {
    if (v == null || v.trim().isEmpty) return 'NIM wajib diisi';
    if (int.tryParse(v.trim()) == null) return 'NIM harus berupa angka';
    if (v.trim().length < 8) return 'NIM minimal 8 digit';
    return null;
  }

  String? validateNama(String? v) {
    if (v == null || v.trim().isEmpty) return 'Nama wajib diisi';
    return null;
  }

  String? validateSemester(String? v) {
    final s = int.tryParse(v?.trim() ?? '');
    if (s == null) return 'Semester wajib berupa angka';
    if (s < 1 || s > 14) return 'Semester antara 1 - 14';
    return null;
  }

  @override
  void onClose() {
    nimC.dispose();
    namaC.dispose();
    semesterC.dispose();
    super.onClose();
  }
}
```

6. Binding

lib/bindings/mahasiswa_binding.dart

```dart
import 'package:get/get.dart';

import '../controllers/mahasiswa_controller.dart';

/// Binding: mendaftarkan dependency (controller) ke GetX
/// sebelum halaman dibuka.
class MahasiswaBinding extends Bindings {
  @override
  void dependencies() {
    Get.lazyPut<MahasiswaController>(() => MahasiswaController());
  }
}
```

7. Nama route

`lib/routes/app_routes.dart`

```dart
/// Daftar nama route aplikasi.
abstract class AppRoutes {
  static const home = '/';
  static const form = '/form';
}
```

8. Pemetaan route

`lib/routes/app_pages.dart`

```dart
import 'package:get/get.dart';

import '../bindings/mahasiswa_binding.dart';
import '../views/mahasiswa_form_view.dart';
import '../views/mahasiswa_list_view.dart';
import 'app_routes.dart';

/// Pemetaan route -> halaman (+ binding).
class AppPages {
  static final pages = [
    GetPage(
      name: AppRoutes.home,
      page: () => const MahasiswaListView(),
      binding: MahasiswaBinding(),
    ),
    GetPage(
      name: AppRoutes.form,
      page: () => const MahasiswaFormView(),
    ),
  ];
}
```

9. View daftar

`lib/views/mahasiswa_list_view.dart`

```dart
import 'package:flutter/material.dart';
import 'package:get/get.dart';

import '../controllers/mahasiswa_controller.dart';

/// VIEW: halaman daftar mahasiswa (READ + tombol Edit/Hapus).
/// `GetView<T>` otomatis menyediakan properti `controller`.
class MahasiswaListView extends GetView<MahasiswaController> {
  const MahasiswaListView({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text('Data Mahasiswa'),
        centerTitle: true,
```

```
      ),
      body: Column(
        children: [
          // ---------- Kolom pencarian ----------
          Padding(
            padding: const EdgeInsets.all(12),
            child: TextField(
              onChanged: (v) => controller.keyword.value = v,
              decoration: const InputDecoration(
                hintText: 'Cari nama atau NIM...',
                prefixIcon: Icon(Icons.search),
                border: OutlineInputBorder(),
              ),
            ),
          ),

          // ---------- Daftar data (reaktif) ----------
          Expanded(
            child: Obx(() {
              final data = controller.filteredList;

              if (data.isEmpty) {
                return const Center(
                  child: Text('Belum ada data mahasiswa'),
                );
              }

              return ListView.builder(
                padding: const EdgeInsets.only(bottom: 80),
                itemCount: data.length,
                itemBuilder: (context, index) {
                  final mhs = data[index];
                  return Card(
                    margin: const EdgeInsets.symmetric(
                        horizontal: 12, vertical: 6),
                    child: ListTile(
                      leading: CircleAvatar(
                        child: Text(mhs.nama[0].toUpperCase()),
                      ),
                      title: Text(mhs.nama),
                      subtitle: Text(
                          'NIM ${mhs.nim}\n${mhs.prodi} • Smt ${mhs.semester}'),
                      isThreeLine: true,
                      trailing: Row(
                        mainAxisSize: MainAxisSize.min,
                        children: [
                          IconButton(
                            visualDensity: VisualDensity.compact,
                            icon: const Icon(Icons.edit, color: Colors.orange),
                            onPressed: () => controller.openEditForm(mhs),
                          ),
                          IconButton(
                            visualDensity: VisualDensity.compact,
                            icon: const Icon(Icons.delete, color: Colors.red),
                            onPressed: () => controller.confirmDelete(mhs),
                          ),
                        ],
                      ),
                    ),
```

```
                );
              },
            );
          }),
        ),
      ],
    ),
    floatingActionButton: FloatingActionButton.extended(
      onPressed: controller.openAddForm,
      icon: const Icon(Icons.add),
      label: const Text('Tambah'),
    ),
  );
 }
}
```

10. View form

lib/views/mahasiswa_form_view.dart

```
import 'package:flutter/material.dart';
import 'package:flutter/services.dart';
import 'package:get/get.dart';

import '../controllers/mahasiswa_controller.dart';

/// VIEW: halaman form untuk CREATE dan UPDATE.
class MahasiswaFormView extends GetView<MahasiswaController> {
  const MahasiswaFormView({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: Text(controller.isEdit ? 'Edit Mahasiswa' : 'Tambah Mahasiswa'),
      ),
      body: Form(
        key: controller.formKey,
        // pesan error diperbarui otomatis saat user mengetik
        autovalidateMode: AutovalidateMode.onUserInteraction,
        child: ListView(
          padding: const EdgeInsets.all(16),
          children: [
            TextFormField(
              controller: controller.nimC,
              keyboardType: TextInputType.number,
              inputFormatters: [FilteringTextInputFormatter.digitsOnly],
              validator: controller.validateNim,
              decoration: const InputDecoration(
                labelText: 'NIM',
                prefixIcon: Icon(Icons.badge),
                border: OutlineInputBorder(),
              ),
            ),
            const SizedBox(height: 16),
            TextFormField(
              controller: controller.namaC,
              textCapitalization: TextCapitalization.words,
```

```
              validator: controller.validateNama,
              decoration: const InputDecoration(
                labelText: 'Nama Lengkap',
                prefixIcon: Icon(Icons.person),
                border: OutlineInputBorder(),
              ),
            ),
            const SizedBox(height: 16),
            DropdownButtonFormField<String>(
              initialValue: controller.selectedProdi.value,
              items: controller.daftarProdi
                  .map((p) => DropdownMenuItem(value: p, child: Text(p)))
                  .toList(),
              onChanged: (v) {
                if (v != null) controller.selectedProdi.value = v;
              },
              decoration: const InputDecoration(
                labelText: 'Program Studi',
                prefixIcon: Icon(Icons.school),
                border: OutlineInputBorder(),
              ),
            ),
            const SizedBox(height: 16),
            TextFormField(
              controller: controller.semesterC,
              keyboardType: TextInputType.number,
              inputFormatters: [FilteringTextInputFormatter.digitsOnly],
              validator: controller.validateSemester,
              decoration: const InputDecoration(
                labelText: 'Semester',
                prefixIcon: Icon(Icons.calendar_month),
                border: OutlineInputBorder(),
              ),
            ),
            const SizedBox(height: 24),
            FilledButton.icon(
              onPressed: controller.save,
              icon: const Icon(Icons.save),
              label: Text(controller.isEdit ? 'Perbarui' : 'Simpan'),
              style: FilledButton.styleFrom(
                padding: const EdgeInsets.symmetric(vertical: 16),
              ),
            ),
          ],
        ),
      ),
    );
  }
}
```

11. Unit test HiveService

`test/hive_service_test.dart`

```
import 'dart:io';

import 'package:crud_mahasiswa/models/mahasiswa_adapter.dart';
import 'package:crud_mahasiswa/models/mahasiswa_model.dart';
```

```dart
import 'package:crud_mahasiswa/services/hive_service.dart';
import 'package:flutter_test/flutter_test.dart';
import 'package:hive_ce/hive_ce.dart';

void main() {
  late Directory dir;
  final service = HiveService();

  setUpAll(() async {
    dir = await Directory.systemTemp.createTemp('hive_test');
    Hive.init(dir.path);
    Hive.registerAdapter(MahasiswaAdapter());
    await Hive.openBox<Mahasiswa>(HiveService.boxName);
  });

  tearDownAll(() async {
    await Hive.deleteFromDisk();
    await dir.delete(recursive: true);
  });

  final budi = Mahasiswa(
      id: '1', nim: '220101001', nama: 'Budi', prodi: 'Informatika', semester: 5);

  test('CREATE & READ', () async {
    await service.create(budi);
    final all = service.getAll();
    expect(all.length, 1);
    expect(all.first.nama, 'Budi');
    expect(all.first.semester, 5);
  });

  test('UPDATE', () async {
    await service.update(budi.copyWith(nama: 'Budi Santoso', semester: 6));
    final all = service.getAll();
    expect(all.length, 1);
    expect(all.first.nama, 'Budi Santoso');
    expect(all.first.semester, 6);
  });

  test('Data tetap ada setelah box ditutup & dibuka lagi', () async {
    await Hive.box<Mahasiswa>(HiveService.boxName).close();
    await Hive.openBox<Mahasiswa>(HiveService.boxName);
    expect(service.getAll().first.nama, 'Budi Santoso');
  });

  test('DELETE', () async {
    await service.delete('1');
    expect(service.getAll(), isEmpty);
  });
}
```

12. Widget test alur CRUD

`test/crud_flow_test.dart`

```dart
import 'dart:io';

import 'package:crud_mahasiswa/main.dart';
```

```dart
import 'package:crud_mahasiswa/models/mahasiswa_adapter.dart';
import 'package:crud_mahasiswa/models/mahasiswa_model.dart';
import 'package:crud_mahasiswa/services/hive_service.dart';
import 'package:flutter/material.dart';
import 'package:flutter_test/flutter_test.dart';
import 'package:get/get.dart';
import 'package:hive_ce/hive_ce.dart';

/// Tap tombol lalu beri waktu nyata agar operasi IO Hive selesai.
Future<void> tapDanTunggu(WidgetTester tester, Finder f) async {
  await tester.tap(f);
  for (var i = 0; i < 5; i++) {
    await tester.pump(const Duration(milliseconds: 300));
    await tester.runAsync(
        () => Future.delayed(const Duration(milliseconds: 100)));
  }
}

void main() {
  testWidgets('Alur CRUD lewat UI', (tester) async {
    await tester.runAsync(() async {
      final dir = await Directory.systemTemp.createTemp('hive_ui');
      Hive.init(dir.path);
      Hive.registerAdapter(MahasiswaAdapter());
      await Hive.openBox<Mahasiswa>(HiveService.boxName);
    });

    await tester.pumpWidget(const MyApp());
    await tester.pumpAndSettle();
    expect(find.text('Belum ada data mahasiswa'), findsOneWidget);

    // CREATE
    await tester.tap(find.text('Tambah'));
    await tester.pumpAndSettle();
    await tester.enterText(find.widgetWithText(TextFormField, 'NIM'), '22010100');
    await tester.enterText(find.widgetWithText(TextFormField, 'Nama Lengkap'), 'Siti');
    await tester.enterText(find.widgetWithText(TextFormField, 'Semester'), '3');
    await tapDanTunggu(tester, find.text('Simpan'));
    await tester.pumpAndSettle(const Duration(seconds: 4));
    expect(find.text('Siti'), findsOneWidget);

    // UPDATE
    await tester.tap(find.byIcon(Icons.edit));
    await tester.pumpAndSettle();
    await tester.enterText(find.widgetWithText(TextFormField, 'Nama Lengkap'), 'Siti Aminah');
    await tapDanTunggu(tester, find.text('Perbarui'));
    await tester.pumpAndSettle(const Duration(seconds: 4));
    expect(find.text('Siti Aminah'), findsOneWidget);

    // DELETE
    await tester.tap(find.byIcon(Icons.delete));
    await tester.pumpAndSettle();
    await tapDanTunggu(tester, find.text('Hapus'));
    await tester.pumpAndSettle(const Duration(seconds: 4));
    expect(find.text('Belum ada data mahasiswa'), findsOneWidget);
    Get.reset();
  });
}
```

14. Kesalahan Umum & Ringkasan

Berikut error yang paling sering muncul saat mahasiswa mengerjakan proyek ini, beserta penyebab dan cara memperbaikinya.

Pesan error / gejala	Penyebab	Solusi
`HiveError: Box not found. Did you forget to call Hive.openBox()?`	`Hive.box()` dipanggil sebelum box dibuka, atau `await` pada `openBox` terlupa	Pastikan `await Hive.open (HiveService.boxName)` a sebelum `runApp`
`HiveError: Cannot read, unknown typeId: 0. Did you forget to register an adapter?`	`registerAdapter` terlupa atau dipanggil setelah `openBox`	Panggil `Hive.registerAdapter(M` sebelum `openBox`
`HiveError: There is already a TypeAdapter for typeId 0.`	Dua adapter berbeda (misalnya `MahasiswaAdapter` dan adapter model baru) memakai typeId yang sama	Beri setiap adapter typeId be daftarkan semuanya di `mair`
Aplikasi crash saat dibuka setelah mengubah adapter (`RangeError`, `type 'int' is not a subtype of type 'String'`)	Urutan atau tipe `read` tidak cocok dengan data lama yang ditulis `write` versi sebelumnya	Samakan urutan `read` dan aplikasi agar file box lama te
`"MahasiswaController" not found. You need to call "Get.put(MahasiswaController())" or "Get.lazyPut(()=>MahasiswaController())"`	Binding tidak terpasang di `GetPage`, atau memakai `MaterialApp` / `home:` bukan `GetMaterialApp` + `initialRoute`	Pastikan `binding: Mahasi` route home dan `main.dart`
Data tersimpan tetapi daftar tidak berubah	`loadData()` tidak dipanggil setelah create/update/delete, atau daftar tidak dibungkus `Obx`	Panggil `loadData()` setela pastikan `controller.filt` dalam `Obx`
`[Get] the improper use of a GetX has been detected`	`Obx` membungkus widget yang tidak membaca satu pun nilai `.obs`	Letakkan `Obx` hanya di seki membaca state reaktif

Pesan error / gejala	Penyebab	Solusi
Tombol Simpan ditekan, snackbar muncul, tapi form tidak tertutup	Memakai `Get.back()` saat masih ada snackbar; GetX hanya menutup snackbar	Gunakan helper `tutup()` y `Get.key.currentState?.`
Form tambah masih berisi data dari edit sebelumnya	Field tidak dikosongkan sebelum membuka form tambah	Panggil `openAddForm()` (y `clear()`) alih-alih `Get.toNamed(AppRoutes.`
Mengedit data membuat data baru (duplikat)	id baru dibuat saat edit sehingga `put` memakai key berbeda	Saat edit, pakai id lama: `id:`
Edit data sendiri ditolak dengan pesan "NIM sudah terdaftar"	Pengecekan duplikat tidak mengecualikan data yang sedang diedit	Tambahkan syarat `m.id !=`
`A TextEditingController was used after being disposed`	Controller GetX dihapus lalu View lama masih memakai field-nya, biasanya karena `Get.delete` manual atau binding dipasang di route form	Pasang binding hanya di rout 7; jangan menghapus contro
`Target of URI doesn't exist: 'package:hive/hive.dart'`	Package yang dipasang `hive_ce` tetapi import masih `hive` (atau sebaliknya)	Samakan import dengan pac `pubspec.yaml` (lihat sectio

Ringkasan modul

- **Model** (`Mahasiswa`) adalah data murni; **TypeAdapter** mengajari Hive cara menulis dan membaca objek tersebut dengan urutan field yang sama.
- `HiveService` menjadi satu-satunya pintu ke database: `put` untuk create dan update, `values` untuk read, `delete` untuk delete.
- `MahasiswaController` menyimpan state reaktif (`.obs`), memvalidasi, mencegah NIM ganda, menyimpan, menghapus, dan memanggil `loadData()` setelah setiap perubahan.
- **View** (`GetView`) hanya menampilkan state lewat `Obx` dan meneruskan klik ke controller. View tidak pernah menyentuh Hive.
- **Binding + Routes** membuat controller otomatis dan memungkinkan navigasi memakai nama.

- Hive harus diinisialisasi dengan urutan: `ensureInitialized` → `initFlutter` → `registerAdapter` → `openBox` → `runApp`.

> ℹ️ **Selanjutnya**
>
> Pola yang sama (model, service, controller, view) dipakai lagi di Modul 13-14. Bedanya, di sana sumber data utama adalah Supabase dan Hive dipakai sebagai cache offline. Jika Anda paham proyek ini, Anda sudah memegang separuh kerangka proyek besar tersebut.

Tugas Praktikum: Praktikum 11B: CRUD Data Mahasiswa dengan Hive + GetX + MVC

Sasaran Praktikum

Membangun ulang aplikasi CRUD Mahasiswa dengan struktur MVC yang benar, lalu mengembangkannya dengan field dan fitur baru tanpa melanggar pemisahan lapisan.

Langkah Pengerjaan

1. **Tugas 1 (Wajib, 25 poin):** Bangun ulang proyek dari section 2-10 sampai berjalan di emulator. Lampirkan screenshot skenario 1-5 dari section 11, termasuk bukti data tetap ada setelah aplikasi ditutup paksa.
2. **Tugas 2 (Wajib, 20 poin):** Tambahkan field `String email` pada model `Mahasiswa`, adapter, form (dengan validasi format email), dan kartu di daftar. Uninstall aplikasi sebelum menjalankan ulang, lalu jelaskan di laporan kenapa langkah ini diperlukan.
3. **Tugas 3 (Wajib, 15 poin):** Tambahkan filter prodi berupa deretan `ChoiceChip` (Semua, Informatika, Sistem Informasi, dan seterusnya) di bawah kolom pencarian. Simpan pilihan di controller sebagai `.obs` dan gabungkan dengan pencarian di `filteredList`.
4. **Tugas 4 (Wajib, 15 poin):** Tampilkan jumlah mahasiswa yang sedang tampil di AppBar, misalnya "Data Mahasiswa (3)", yang ikut berubah saat data ditambah, dihapus, atau dicari.
5. **Tugas 5 (Wajib, 15 poin):** Tambahkan halaman detail mahasiswa (route `/detail`) yang terbuka saat kartu diketuk, berisi semua field dan tombol Edit/Hapus. Daftarkan route di `AppPages`.
6. **Tugas 6 (Wajib, 10 poin):** Tambahkan minimal 2 unit test baru di `hive_service_test.dart` (misalnya menyimpan 3 data lalu memastikan `getAll` berisi 3, dan menghapus id yang tidak ada tidak menyebabkan error). Semua test harus lulus.
7. **Bonus (+10 poin):** Ganti tombol hapus dengan gestur geser `Dismissible` dan snackbar berisi aksi "Batalkan" yang mengembalikan data yang terhapus lewat `HiveService.create`.

Output yang Diharapkan

Aplikasi CRUD Mahasiswa yang berjalan offline dengan field email, filter prodi, jumlah data di AppBar, halaman detail, serta data yang tetap ada setelah restart. Disertai output flutter analyze tanpa issue dan flutter test yang semua lulus.

Rubrik Penilaian

Tugas 1: kelengkapan CRUD dan bukti persistensi (25%). Tugas 2: field baru end-to-end beserta validasi (20%). Tugas 3: filter prodi reaktif (15%). Tugas 4: jumlah data reaktif (15%). Tugas 5: halaman detail dan route (15%). Tugas 6: unit test (10%). Bonus Dismissible + Batalkan (+10%). Pengurangan nilai jika View memanggil Hive secara langsung, memakai setState untuk data mahasiswa, atau flutter analyze masih menampilkan issue.

Kuis Pemahaman

1. Pada proyek ini, file mana yang SATU-SATUNYA boleh memanggil Hive.box() untuk operasi CRUD?

A. mahasiswa_list_view.dart

B. mahasiswa_controller.dart

C. hive_service.dart

D. app_pages.dart

2. Kenapa HiveService.create dan HiveService.update sama-sama memakai _box.put(mhs.id, mhs)?

A. Karena Hive tidak punya cara untuk mengubah data

B. Karena put membuat data baru jika key belum ada dan menimpa data lama jika key sudah ada

C. Karena update harus menghapus data dulu lalu menambah ulang

D. Karena put selalu membuat key acak

3. Di MahasiswaAdapter, write menulis id, nim, nama, prodi, semester. Apa akibatnya jika read ditulis membaca nama sebelum nim?

A. Tidak ada akibat, Hive mencocokkan berdasarkan nama field

B. Hive otomatis mengurutkan ulang field

C. Nilai nim dan nama tertukar saat data dibaca, karena adapter manual hanya mengandalkan urutan

D. Aplikasi tidak bisa dikompilasi

4. Manakah urutan yang benar di main() sebelum runApp?

A. Hive.openBox, Hive.registerAdapter, Hive.initFlutter

B. WidgetsFlutterBinding.ensureInitialized, Hive.initFlutter, Hive.registerAdapter, Hive.openBox

C. Hive.registerAdapter, runApp, Hive.openBox

D. Hive.initFlutter, Hive.openBox, Hive.registerAdapter

5. Setelah menyimpan data, controller memanggil loadData(). Apa yang terjadi jika baris ini dihapus?

A. Data tidak tersimpan ke Hive

B. Data tersimpan di Hive, tetapi daftar di layar tidak berubah sampai aplikasi dibuka ulang

C. Aplikasi crash

D. Hive otomatis memperbarui Obx

6. Bagaimana satu halaman MahasiswaFormView bisa dipakai untuk tambah dan edit?

A. Dengan membuat dua controller berbeda

B. Controller menyimpan editingId: null berarti mode tambah, berisi id berarti mode edit; judul, tombol, dan proses simpan menyesuaikan isEdit

C. Dengan mengirim parameter lewat constructor View

D. Dengan memakai dua route yang menunjuk widget berbeda

7. Saat mengedit Budi tanpa mengubah NIM, kenapa pengecekan NIM ganda tidak menolak penyimpanan?

A. Karena pengecekan hanya dilakukan saat mode tambah

B. Karena syarat m.id != editingId mengecualikan data yang sedang diedit

C. Karena NIM disimpan sebagai int

D. Karena Hive menolak key ganda secara otomatis

8. Kenapa MahasiswaBinding hanya dipasang pada route home dan tidak pada route form?

A. Karena route form tidak boleh memakai GetX

B. Karena form selalu dibuka dari halaman daftar sehingga controller yang sama sudah ada di memori, dan form harus membaca data yang disiapkan openEditForm

C. Karena Binding hanya boleh dipakai sekali di seluruh aplikasi

D. Karena route form memakai StatefulWidget

9. Kenapa unit test memakai Hive.init(dir.path) dan bukan Hive.initFlutter()?

A. Karena Hive.initFlutter lebih lambat

B. Karena saat test tidak ada perangkat sehingga folder penyimpanan ditentukan sendiri di folder sementara, dan data test tidak bercampur dengan data aplikasi

C. Karena Hive.init mengenkripsi data

D. Karena Hive.initFlutter tidak mendukung TypeAdapter

Kunci Jawaban & Pembahasan

1. C — HiveService adalah lapisan akses data. Controller memanggil service, dan View hanya memanggil controller. Dengan begitu, jika penyimpanan diganti (misalnya ke Supabase), cukup HiveService yang diubah.

2. B — put dengan key yang sama menimpa value lama. Karena id dipakai sebagai key, mengedit data cukup dengan put memakai id lama.

3. C — Adapter manual tidak menyimpan nama field. Data dibaca persis sesuai urutan penulisan, sehingga urutan read harus sama dengan urutan write.

4. B — Binding harus siap sebelum memakai plugin, Hive harus tahu folder penyimpanan, adapter harus terdaftar sebelum box berisi objek Mahasiswa dibuka, dan box dibuka sebelum controller memakainya.

5. B — Obx mendengarkan mahasiswaList (RxList), bukan isi box. loadData() mengisi ulang mahasiswaList dengan assignAll sehingga Obx membangun ulang daftar.

6. B — openAddForm mengosongkan editingId dan field, openEditForm mengisinya dengan data lama. save() memakai isEdit untuk memilih update atau create dan id lama atau id baru.

7. B — any(...) mencari mahasiswa lain dengan NIM sama. Data milik sendiri dikecualikan lewat perbandingan id, sehingga edit tanpa mengubah NIM tetap diizinkan.

8. B — Halaman daftar dan form berbagi satu MahasiswaController. Data field form diisi controller sebelum pindah halaman, lalu dibaca oleh View form lewat Get.find.

9. B — Hive.initFlutter mencari folder dokumen aplikasi lewat plugin, yang tidak tersedia saat flutter test. Folder sementara juga mudah dihapus di tearDownAll.